



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Opracowanie elektronicznego optymalizatora do sterowania pracą pompy.

Paweł KRUTAK

Nr albumu: 300740

Kierunek: Automatyka i Robotyka

Specjalność: Technologie informacyjne w Automatyce i Robotyce

PROWADZĄCY PRACĘ

dr inż. Grychowski Tomasz

KATEDRA Pomiarów i Systemów Sterowania

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Opracowanie elektronicznego optymalizatora do sterowania pracą pompy.

Opracowanie elektronicznego optymalizatora do sterowania pracą pompy.

Streszczenie

Praca inżynierska dotyczy opracowania i budowy elektronicznego (mikroprocesorowego) modelu zadajnika kontrolującego wydajność pompy. Urządzenie zostanie zrealizowane na bazie wybranego mikrokontrolera z zaprojektowanymi odpowiednimi interfejsami wejścia/wyjścia do współpracy z urządzeniami zewnętrznymi oraz komunikacji z użytkownikiem.

Zasadę działania systemu można opisać w następujący sposób: zadajnik odbiera sygnał analogowy 0 – 10 V z zaworu regulacyjnego reprezentujący położenie kuli zaworu. Następnie porównuje tę wartość z nastawą wyjściową pompy. W przypadku, gdy sygnał z zaworu jest mniejszy od wartości zadanej, urządzenie zmniejsza sygnał wyjściowy o 0,1 V w regularnych odstępach czasu określonych za pomocą encodera aż do osiągnięcia zadanej wartości. Analogicznie, jeśli wartość położenia kuli zaworu przekracza wartość zadaną, zadajnik zwiększa sygnał wyjściowy o 0,1 V w określonych odstępach czasu.

Układ został wyposażony w trzy encodery umożliwiające konfigurację parametrów pracy, dwa wejścia analogowe 0 – 10 V oraz jedno wyjście analogowe 0 – 10 V, co zapewnia precyzyjne sterowanie i dostosowanie działania pompy do wymagań operacyjnych. Urządzenie pracuje w dwóch trybach: automatycznym oraz manualnym.

W ramach pracy przeprowadzone zostaną również testy funkcjonalne z wykorzystaniem dostępnych urządzeń pomiarowych oraz zadajników napięć, mające na celu weryfikację poprawności działania i zgodności z założeniami projektowymi. Zadanie to zrealizowane jest według wytycznych i na potrzeby firmy BELIMO - BELIMO Siłowniki S.A.

Słowa kluczowe

(zadajnik, urządzenie elektroniczne, sterowanie pompą, układ mikroprocesorowy, automatyka budynkowa)

Thesis title

Development of an electronic optimizer to control pump operation.

Abstract

The engineering thesis focuses on the development and construction of an electronic (microprocessor-based) model of a setpoint controller for regulating pump performance. The device will be implemented using a selected microcontroller with appropriately designed input/output interfaces to interact with external devices and facilitate user

communication. The system's operating principle can be described as follows: the controller receives an analog 0 – 10 V signal from a regulating valve, representing the position of the valve ball. It then compares this value with the pump's output setpoint. If the signal from the valve is lower than the setpoint, the device reduces the output signal by 0.1 V at regular time intervals determined by an encoder until the target value is reached. Conversely, if the valve ball position value exceeds the setpoint, the controller increases the output signal by 0.1 V at the specified time intervals. The system is equipped with three encoders for configuring operating parameters, two analog inputs 0 – 10 V, and one analog output 0 – 10 V, ensuring precise control and adaptation of pump performance to operational requirements. The device operates in two modes: automatic and manual. As part of the thesis, functional tests will be conducted using available measurement instruments and voltage generators to verify the correct operation and compliance of the device with the design assumptions. The task is carried out in accordance with the guidelines and for the purposes of BELIMO - BELIMO Siłowniki S.A.

Key words

(transmitter, electronic device, pump control, microprocessor system, building automation)

Spis treści

1	Wstęp	1
2	Opis działania zadajnika.	4
2.1	Zastosowania i uzasadnienie budowy zadajnika.	4
2.1.1	Przykład konkretnej instalacji.	5
2.2	Wymagania sprzętowe do projektu.	6
2.3	Opis urządzeń do których zadajnik ma zostać podpięty.	7
2.4	Dokładny opis działania systemu.	8
2.5	Instrukcja obsługi elektronicznego optymalizatora.	9
3	Wykorzystane narzędzia pomiarowe.	11
3.1	Zasilacz laboratoryjny.	11
3.2	Wykorzystane multimetry:	12
3.2.1	Multimetr Sonel	13
4	Hardware	16
4.1	Schemat połączeń	16
4.2	Opis komponentów.	17
4.2.1	Mikrokontroler ESP32-DevKit v1.	17
4.2.2	Moduł RTC DS1307 z pamięcią EEPROM 24C32.	18
4.2.3	Wyświetlacz OLED 1.3"(I ² C).	18
4.2.4	Encoder obrotowy KY-040.	18
4.2.5	Przetwornica DC-DC MP2315	19
4.2.6	Wzmacniacz operacyjny LM358P	19
4.2.7	Elementy pasywne	19
4.2.8	Diody LED	19
4.3	Kosztorys	20
4.4	Wybór Komponentów.	20
4.4.1	Zmiana napięcia wejściowego.	20
4.4.2	Zmiana napięcia wyjściowego.	23
4.4.3	Wybór Przetwornicy Impulsowej do Konwersji Napięcia	24

4.4.4	Wybór Encoderów Zamiast Potencjometrów do Zadawania Wartości.	26
4.4.5	Wskaźniki LED na Wejściach i Wyjściu Układu	26
4.4.6	Wyświetlacz do Wizualizacji Parametrów Układu i Czasu	26
4.5	Projektowanie oraz wykonanie płytki drukowanej.	26
4.6	Projektowanie i wydruk obudowy.	30
4.7	Końcowy wygląd elektronicznego optymalizatora.	32
5	Software	33
5.1	Kod zadajnika.	34
5.1.1	Opis wykorzystanych bibliotek.	34
5.1.2	Uzasadnienie zastosowania bibliotek.	35
5.1.3	Opis zdefiniowanych pinów	35
5.1.4	Konfiguracja wyświetlacza OLED	37
5.1.5	Konfiguracja modułu RTC.	38
5.1.6	Definicje trybów pracy i źródeł wartości	38
5.1.7	Funkcja korekty offsetu napięcia.	39
5.1.8	Funkcja interpolacji kawałkowej.	40
5.1.9	Funkcja wygładzania eksponencjalnego.	41
5.1.10	Funkcja zmiany trybu pracy systemu.	43
5.1.11	Funkcja obsługi encodera BMS.	44
5.1.12	Funkcja obsługi encodera VOLT.	44
5.1.13	Funkcja obsługi encodera TIME.	45
5.1.14	Inicjalizacja interfejsów komunikacyjnych i urządzeń peryferyjnych .	46
5.1.15	Sprawdzenie obecności modułu RTC.	46
5.1.16	Synchronizacja czasu modułu RTC.	47
5.1.17	Inicjalizacja wyświetlacza OLED i komunikat powitalny.	47
5.1.18	Konfiguracja trybów pracy pinów mikrokontrolera.	48
5.1.19	Inicjalizacja stanów początkowych encoderów.	48
5.1.20	Konfiguracja przerw dla encoderów.	49
5.1.21	Inicjalizacja początkowego stanu wyjściowego pompy.	49
5.1.22	Obsługa przycisku z eliminacją drgań styków (<i>debouncing</i>).	50
5.1.23	Odczyt, przetwarzanie i skalowanie napięć z pinów: VP i VN. . . .	51
5.1.24	Logika regulacji w trybie automatycznym.	51
5.1.25	Logika regulacji w trybie ręcznym.	53
5.1.26	Obsługa diod sygnalizacyjnych.	53
5.1.27	Wyświetlanie informacji na wyświetlaczu OLED.	54
5.1.28	Realizacja operacji w regularnych odstępach czasu.	55
5.2	Podsumowanie.	56

6	Weryfikacja i walidacja	57
6.1	Weryfikacja i walidacja	57
6.2	Badanie liniowości przetwarzanego napięcia z wejść i wyjścia elektronicznego optymalizatora.	57
6.2.1	Badanie liniowości przetwarzanego napięcia sygnału wyjściowego ze wzmacniacza.	58
6.2.2	Badanie liniowości przetwarzanego napięcia sygnału z położenia kuli zaworu EV.	58
6.2.3	Badanie liniowości przetwarzanego napięcia sygnału BMS.	59
6.3	Pomiary napięcia referencyjnego i rzeczywistego dla wejść zadajnika. . . .	60
6.3.1	Pomiary napięcia referencyjnego i rzeczywistego dla EV.	60
6.3.2	Pomiary napięcia referencyjnego i rzeczywistego dla wejścia z systemu zarządzania budynkiem.	62
6.3.3	Pomiary napięcia referencyjnego i rzeczywistego dla wyjścia.	63
7	Podsumowanie i wnioski	65
	Bibliografia	68
	Spis skrótów i symboli	70
	Lista dodatkowych plików, uzupełniających tekst pracy	71
	Spis rysunków	73
	Spis tabel	74

Rozdział 1

Wstęp

Praca inżynierska koncentruje się na opracowaniu i budowie elektronicznego modelu zadajnika kontrolującego wydajność pompy, wykorzystującego mikrokontroler jako jednostkę obliczeniową. Celem projektu jest zbudowanie prototypu urządzenia według wytycznych i na potrzeby firmy BELIMO - BELIMO Siłowniki S.A. , który może być zastosowany do pracy z rzeczywistymi urządzeniami przemysłowymi. Kluczowym zadaniem opracowanego urządzenia jest precyzyjna regulacja parametrów pracy pompy, co pozwoli na optymalizację jej wydajności i dostosowanie do zmiennych warunków operacyjnych. Przez optymalizację rozumiemy dynamiczne dostosowanie zadajnika do zmieniających się parametrów wejściowych. Przez takie podejście uzyskujemy poprawę efektywności i wydajności pracy pompy. Niniejsza praca składa się z siedmiu rozdziałów, w których szczegółowo opisano proces projektowania, implementacji oraz testowania elektronicznego optymalizatora do sterowania pracą pompy.

Rozdział pierwszy, **Wstęp**, przedstawia cel projektu, znaczenie podjętego tematu oraz spodziewane korzyści wynikające z realizacji projektu. Wyjaśniono również kontekst przemysłowy, w którym może zostać zastosowany zaprojektowany zadajnik.

W rozdziale drugim, **Opis działania zadajnika**, omówiono: zastosowanie i uzasadnienie budowy zadajnika, przykład konkretnej instalacji, wymagania sprzętowe oraz urządzenia, z którymi zadajnik będzie współpracował. Szczegółowo zaprezentowano zasadę działania systemu, uwzględniając różne scenariusze pracy w zależności od dostępności sygnałów sterujących. Zadajnik będzie wyposażony w następujące wejścia i wyjście.

Wejścia: dwa wejścia analogowe 0 – 10 V (jedno z pompy, drugie z zaworu EV).

Wyjście: jedno wyjście analogowe 0 – 10 V (do sterowania pompy).

Nastawy: encoder z wartościami od 0 do 10 V odpowiadający wartości wyjściowej. Encoder z wartościami 0 – 10 V w przypadku braku informacji z systemu zarządzania budynkiem "BMS"(ang. *Building Management System*), gdzie sygnał pochodzi ze sterownika PLC (ang. *Programmable Logic Controller*). Podsystemu automatyki budynkowej, który wypracował wyjście na podstawie stanu obiektu i przesłał je do

urządzenia w postaci wartości analogowej w standardowym zakresie 0 – 10 V oraz encoder do ustawiania czasu. Zadajnik pobiera informacje z zaworu regulacyjnego, który wysyła sygnał analogowy 0 – 10 V, reprezentujący położenie kuli zaworu. Na podstawie tego sygnału zadajnik porównuje otrzymaną wartość z wartością wyjściową. Jeśli sygnał z zaworu jest mniejszy od wartości wyjściowej, zadajnik zmniejsza sygnał wyjściowy do pompy o 0,1 V w ustalonych odstępach czasu, aż do osiągnięcia wartości zadanej. Analogicznie, jeśli wartość położenia kuli zaworu jest większa od wyjściowej, zadajnik zwiększa sygnał wyjściowy pompy o 0,1 V w tych samych odstępach czasu. Zawór Energy Valve [5] generuje sygnał wyjściowy 0 – 10 V, który można skonfigurować do przesyłania informacji o położeniu kuli zaworu. Zadajnik precyzuje pracę pompy na podstawie stopnia otwarcia kuli zaworu, dążąc do utrzymania pożądanej pracy instalacji na podstawie wartości wejściowej. W przypadku braku sygnału z BMS (Building Management System), zadajnik wykorzystuje ustawioną wartość początkową obrotów pompy jako wartość domyślną. W sytuacji, gdy sygnał z BMS jest dostępny, zadajnik musi uwzględnić ten sygnał jako nadrzędny, co oznacza, że wszelkie inne operacje są przerywane w przypadku zmiany sygnału z BMS. Przy założeniu braku sygnału z BMS, początkowa nastawa obrotów pompy ustawiona jest na 100%. Jeśli zawór Belimo wskazuje położenie kuli na 30% (3 V), zadajnik co ustawioną przez użytkownika nastawę czasu 5, 15, 30, 45, 60 sekund zmniejsza sygnał wyjściowy do pompy o 0,1 V, dążąc do osiągnięcia 30% obrotów pompy. Proces ten trwa do momentu, aż sygnał z zaworu nie osiągnie zadanej wartości, po czym zadajnik utrzymuje odpowiednie obroty pompy, do momentu zmiany sygnału zadanego. Na końcu tego rozdziału opisana została instrukcja obsługi dla użytkownika elektornicznego optymalizatora.

Rozdział trzeci, **Wykorzystane narzędzia pomiarowe**, zawiera opis przyrządów używanych podczas testowania, takich jak zasilacz laboratoryjny oraz multimetry, ze szczególnym uwzględnieniem ich parametrów i zastosowania.

Rozdział czwarty, **Hardware**, skupia się na sprzętowej części projektu. Zawarto w nim opis komponentów wykorzystanych w zadajniku, w tym mikrokontrolera ESP32, modułu RTC, wyświetlacza OLED oraz innych elementów elektronicznych. Szczególną uwagę poświęcono projektowaniu układu elektronicznego, w tym schematu połączeń, płytki drukowanej oraz obudowy urządzenia, co pozwala na pełne zrozumienie poszczególnych etapów realizacji projektu. Dodatkowo, omówiono kompatybilność wybranych komponentów z mikrokontrolerem oraz pozostałymi elementami układu, co pozwoli na eliminację potencjalnych problemów w fazie implementacji. Rozdział czwarty dostarcza kompleksowego spojrzenia na projektowanie i implementację warstwy sprzętowej, podkreślając znaczenie synergii pomiędzy komponentami elektronicznymi a oprogramowaniem sterującym. Opisane procesy projektowe i techniczne decyzje stanowią solidną podstawę dla realizacji funkcjonalnego i niezawodnego zadajnika, odpowiadającego wymaganiom założonym na etapie planowania projektu.

Rozdział piąty, **Software**, obejmuje szczegółowy opis oprogramowania zadajnika, napisanego w języku C z wykorzystaniem środowiska Arduino IDE. Oprogramowanie stanowi kluczowy element projektu, umożliwiającą pełną kontrolę nad sprzętowymi komponentami urządzenia oraz realizację jego funkcji użytkowych. Początkowo przedstawiono strukturę programu, w tym podział na moduły oraz funkcje realizujące konkretne zadania. Omówiono główne bloki kodu, takie jak inicjalizacja urządzeń peryferyjnych, obsługa wejść/wyjść cyfrowych i analogowych, a także implementacje kluczowych algorytmów sterujących. Szczególną uwagę poświęcono usprawnieniu kodu, aby zapewnić jego efektywność i zgodność z ograniczeniami zasobów mikrokontrolera. Dalsza część rozdziału zostanie poświęcona omówieniu zastosowanych bibliotek, takich jak `Wire.h`, `U8g2lib.h`, `RTClib.h` oraz innych, które wspierają komunikację z urządzeniami peryferyjnymi. Szczegółowo opisane zostaną sposoby wykorzystania funkcji tych bibliotek do realizacji takich zadań, jak sterowanie wyświetlaczem OLED, synchronizacja czasu z modułem RTC, czy obsługa interfejsu I²C. Wyjaśnione zostaną mechanizmy, dzięki którym biblioteki te zostały zaadaptowane do specyficznych wymagań projektu. Ponadto omówione zostaną metody obsługi zdarzeń, w tym przerwania oraz reakcje na sygnały wejściowe generowane przez przyciski. Szczególną uwagę poświęcono implementacji różnych trybów pracy urządzenia, w tym automatycznego oraz ręcznego sterowania. W końcowej części rozdziału zostaną omówione metody testowania i weryfikacji poprawności działania kodu. Opisane zostaną techniki debugowania oraz sposoby wykrywania i eliminacji potencjalnych błędów. Rozdział ten ma na celu dostarczenie pełnego obrazu procesu tworzenia i implementacji oprogramowania, które zapewnia niezawodne i efektywne działanie zadajnika.

Rozdział szósty, **Weryfikacja i walidacja**, prezentuje fazę testowania, podczas której zostanie sprawdzona poprawność działania zadajnika z wykorzystaniem dostępnych urządzeń pomiarowych oraz zadajników napięć. Testy te mają na celu potwierdzenie, że zadajnik działa zgodnie z założeniami i jest w stanie skutecznie zarządzać pracą pompy w różnych scenariuszach operacyjnych. W dalszych częściach pracy zostaną przedstawione szczegółowe opisy poszczególnych funkcjonalności, wyniki testów oraz analiza działania zadajnika w kontekście rzeczywistych aplikacji przemysłowych.

Ostatni rozdział, **Podsumowanie i wnioski**, podsumowuje efekty pracy, ocenia stopień realizacji założonych celów oraz wskazuje potencjalne kierunki rozwoju projektu.

Rozdział 2

Opis działania zadajnika.

Współczesne systemy automatyzacji wymagają coraz bardziej zaawansowanych narzędzi umożliwiających precyzyjne sterowanie pracą urządzeń, w tym pomp wykorzystywanych w różnorodnych instalacjach przemysłowych, budynkowych i technologicznych. Zadania te obejmują dynamiczne dostosowywanie parametrów pracy do zmieniających się warunków operacyjnych oraz potrzeb użytkowników.

Rozdział drugi pracy inżynierskiej koncentruje się na szczegółowym opisie działania projektowanego zadajnika, jego funkcjonalności oraz współpracy z elementami systemu. Zadania realizowane przez zadajnik obejmują precyzyjną regulację pracy pompy w instalacjach technicznych, takich jak systemy HVAC, budynki komercyjne, instalacje przemysłowe oraz systemy chłodzenia serwerowni.

W kolejnych podrozdziałach przedstawiono szerokie zastosowania zadajnika, a także omówiono przykłady konkretnych instalacji, w których może być on wykorzystany. Zaprezentowano także szczegółowe wymagania sprzętowe, opis urządzeń współpracujących z zadajnikiem oraz dokładny opis zasad jego działania.

2.1 Zastosowania i uzasadnienie budowy zadajnika.

Projektowany zadajnik znajduje szerokie zastosowanie w systemach automatyzacji, które wymagają precyzyjnego sterowania pracą pomp oraz usprawnienia przepływu medium. Przykłady takich instalacji obejmują:

- **Systemy HVAC (Heating, Ventilation, Air Conditioning):** Zadaniem zadajnika w takich systemach jest dynamiczne dostosowanie pracy pomp obiegowych do aktualnych warunków eksploatacyjnych, takich jak zmienne zapotrzebowanie na ciepło lub chłód. Dzięki precyzyjnej regulacji przepływu medium możliwe jest zwiększenie efektywności energetycznej całego systemu, co prowadzi do obniżenia kosztów eksploatacji oraz zmniejszenia emisji CO₂.
- **Budynki komercyjne i użyteczności publicznej:** W dużych budynkach,

takich jak biurowce, hotele czy szpitale, utrzymanie stabilnych parametrów pracy układów cyrkulacyjnych jest kluczowe dla komfortu użytkowników oraz wydajności systemu. Zadania realizowane przez zadajnik, takie jak dostosowanie prędkości pomp do sygnałów wejściowych z systemu zarządzania budynkiem (BMS), zapewniają niezawodność i efektywność instalacji.

- **Instalacje przemysłowe:** W procesach technologicznych wymagających precyzyjnej regulacji przepływu medium, np. w układach chłodzenia maszyn lub urządzeń, zadajnik umożliwia sterowanie pracą pompy, co przekłada się na większą kontrolę nad procesami oraz oszczędności energetyczne.
- **Systemy chłodzenia serwerowni:** W serwerowniach precyzyjne sterowanie temperaturą jest kluczowe dla zapewnienia ciągłości pracy sprzętu. Zadajnik umożliwia regulację przepływu medium chłodzącego w zależności od obciążenia systemu, co pozwala na efektywne zarządzanie temperaturą.

Głównym celem projektu zadajnika jest zwiększenie efektywności i elastyczności zarządzania pracą pomp w różnych instalacjach. Współczesne systemy HVAC oraz instalacje przemysłowe wymagają coraz bardziej zaawansowanych metod regulacji, które pozwalają na dynamiczne dostosowywanie parametrów pracy do zmieniających się warunków.

2.1.1 Przykład konkretnej instalacji.

Projektowany zadajnik może być zastosowany w instalacji grzewczej nowoczesnego budynku biurowego, w którym system HVAC (ogrzewanie, wentylacja i klimatyzacja) wykorzystuje pompy obiegowe do cyrkulacji medium grzewczego.

W tej instalacji zastosowano:

- **Pompy obiegowe Wilo-Yonos ECO 25/1-5**, sterowane napięciem 0 – 10 V, które dynamicznie dostosowują prędkość przepływu wody w zależności od zapotrzebowania na ciepło w różnych strefach budynku.
- **Zawory Belimo Energy Valve™**, które monitorują i kontrolują przepływ medium oraz przekazują sygnały o położeniu kuli zaworu (stopnia otwarcia). Położenie kuli zaworu jest kluczowym parametrem, który pozwala na precyzyjne określenie bieżącego zapotrzebowania na przepływ medium w danej strefie budynku.
- **System zarządzania budynkiem (BMS)**, który zbiera dane o zapotrzebowaniu na energię cieplną w poszczególnych strefach obiektu i przesyła je do zadajnika jako sygnały wejściowe.

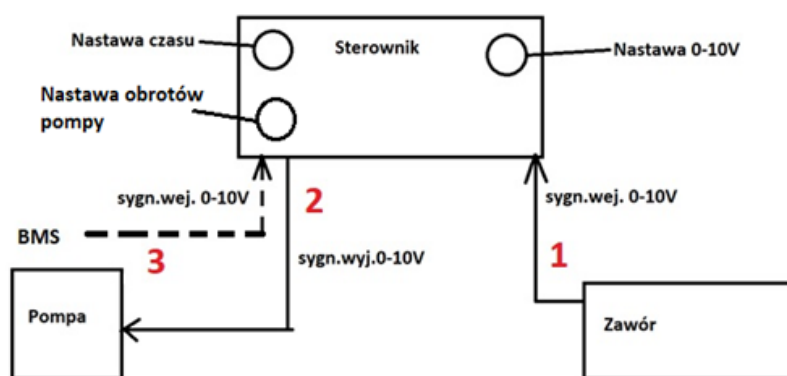
W praktyce zadajniki odczytują sygnały z zaworu regulacyjnego oraz z systemu BMS. Na podstawie informacji o położeniu kuli zaworu oraz sygnału z BMS, zadajniki dynamicznie regulują prędkość obrotową pomp. Dzięki temu każda strefa budynku otrzymuje dokładnie tyle energii cieplnej, ile potrzebuje, co minimalizuje straty energetyczne i obniża koszty eksploatacji.

Przykład zastosowania: W budynku o powierzchni 10 000 m², podzielonym na kilka stref grzewczych (np. biura, sale konferencyjne, pomieszczenia techniczne), zadajniki umożliwiają regulację pracy pomp w taki sposób, aby dopasować przepływ medium grzewczego do zapotrzebowania każdej strefy. Na przykład:

- W godzinach pracy biur (8:00–18:00) system, bazując na informacji o położeniu kuli zaworu, zwiększa przepływ medium do stref intensywnie użytkowanych.
- W godzinach nocnych system BMS, przy wsparciu zadajnika, redukuje przepływ do minimalnego poziomu, pozwalając jedynie na utrzymanie temperatury ochronnej (np. 15°C).

Dzięki wykorzystaniu informacji o położeniu kuli zaworu oraz sygnałów z systemu BMS, takie rozwiązanie pozwala na wydajniejszą pracę systemu grzewczego, zmniejszenie zużycia energii o 20–30% oraz zwiększenie komfortu użytkowników budynku.

2.2 Wymagania sprzętowe do projektu.



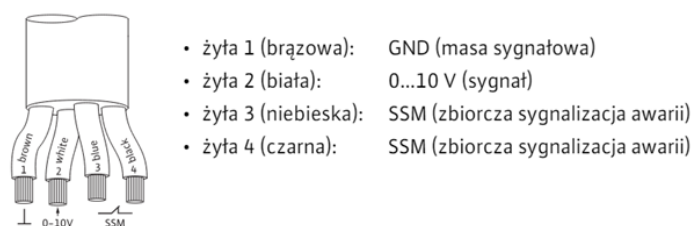
Rysunek 2.1: Ideowy schemat działania zadajnika (rysunek od "zleceńodawcy").

Zaprojektowany zadajnik ma wykorzystywać mikrokontroler jako centralną jednostkę obliczeniową, która ma odczytywać wejścia i w odpowiedni sposób wysterowywać wyjścia. Na rysunku 2.1 przedstawiono ideowy schemat działania układu. Mikrokontroler odczytuje sygnały analogowe z zakresu 0 – 10 V. Jeden z nich przekazuje

informacje o położeniu kuli zaworu EV, a drugi dostarcza dane o prędkości obrotowej silnika pompy - sygnał z systemu zarządzania budynkiem "BMS"(ang. *Building Management System*). Wyjście przekazuje sygnał ciągły 0 – 10 V, który steruje pompą. Na rysunku 2.1 przedstawiono trzy encodery umożliwiające ustalenie nastaw. Pierwszy z nich to nastawa czasu, w którym użytkownik może wybrać wartości z przedziału 5, 15, 30, 45, 60 sekund. Drugi encoder jest do ustawienia obrotów pompy w przypadku braku sygnału wejściowego z BMS'u - nastawa wartości 0 – 10 V z krokiem 0.1 V. Ostatni z kolei ma działać tak samo jak drugi encoder, ale służy do ustawienia wartości wyjściowej do sterowania pompą.

2.3 Opis urządzeń do których zadajnik ma zostać podpięty.

Pompa obiegowa Wilo-Yonos ECO 25/1-5 BMS z sygnałem 0-10V.



Rysunek 2.2: Opis sygnałów i wyprowadzeń pompy obiegowej Wilo-Yonos ECO 25 [22].

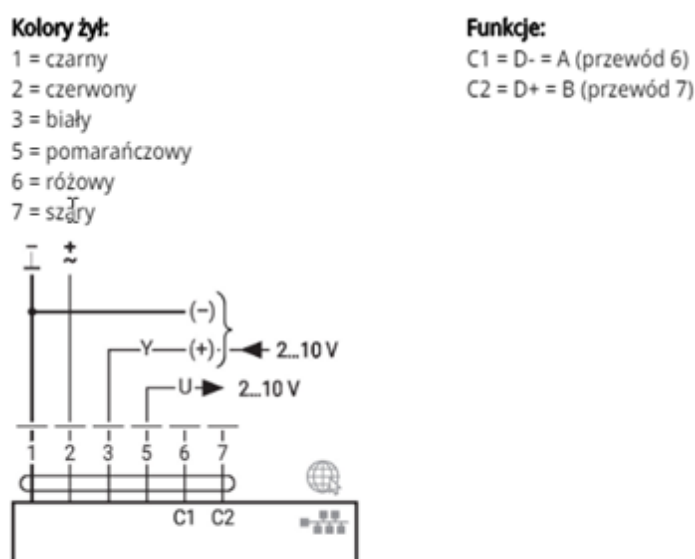
Pompa obiegowa Wilo-Yonos ECO 25/1-5 to wydajne urządzenie wykorzystywane w systemach grzewczych i chłodniczych, które pracuje w zakresie napięcia sterującego 0 – 10 V. Posiada zdolność dostosowywania swoich obrotów w zależności od sygnału sterującego, co pozwala na polepszenie pracy całego systemu. Sygnał sterujący można przesyłać do pompy za pomocą dedykowanego wejścia - rysunek 2.2, co umożliwia jej integrację z systemami automatyki budynkowej. Masa sygnału (GND) oraz zasilanie pompy muszą być poprawnie podłączone, aby zapewnić właściwą komunikację i stabilną pracę urządzenia [22].

Sygnał 0 – 10 V w technologii automatyki budynkowej (Building Management System, BMS) jest standardowym sygnałem analogowym, powszechnie stosowanym w systemach automatyki przemysłowej i budynkowej. W praktyce sygnał ten może być generowany przez sterownik PLC (ang. Programmable Logic Controller) będący częścią podsystemu automatyki budynkowej. Sterownik, na podstawie analizy stanu obiektu (np. odczytu temperatury, przepływu czy zapotrzebowania na energię cieplną), generuje sygnał wyjściowy i przesyła go do naszego urządzenia w postaci standardowego sygnału

analogowego 0 – 10 V. Taki sygnał może być wykorzystany do sterowania parametrami pracy urządzeń, takimi jak prędkość obrotowa pompy czy przepływ medium. Dzięki temu możliwe jest precyzyjne zarządzanie systemami grzewczymi i chłodniczymi, co prowadzi do zwiększenia efektywności energetycznej oraz redukcji kosztów eksploatacji.

Zawór Belimo Energy Valve™.

Zawór Belimo Energy Valve™ to zaawansowane urządzenie służące do zarządzania przepływem medium w systemach HVAC. Jest wyposażony w interfejs sygnału 0 – 10 V co umożliwia monitorowanie i kontrolę położenia kuli zaworu (stopnia otwarcia). Funkcjonalność ta pozwala na precyzyjne sterowanie wydajnością instalacji, zwiększając jej efektywność. *Energy Valve* (EV) pozwala także na wybór rodzaju przesyłanej informacji. Jednym z najistotniejszych parametrów jest pozycja kuli zaworu, która może służyć za podstawę do regulacji pracy pompy. Informację o położeniu kuli zaworu otrzymujemy z pomarańczowego oraz białego przewodu przedstawionego na rysunku 2.3, Urządzenie wspiera działania usprawniające, dzięki czemu możliwe jest utrzymywanie odpowiednich parametrów instalacji przy minimalnym zużyciu energii [5].



Rysunek 2.3: Wyprowadzenia sygnałów z zaworu Belimo Energy Valve™ [5].

2.4 Dokładny opis działania systemu.

System sterowania pompą na podstawie informacji z zaworu oparty jest na mikrokontrolerze ESP32 i umożliwia dynamiczne dostosowywanie pracy instalacji w zależności od aktualnych warunków oraz preferencji użytkownika. Głównie zadanie

systemu polega na regulacji prędkości pompy na podstawie sygnałów napięciowych 0 – 10 V pochodzących z zaworu regulacyjnego i systemu zarządzania budynkiem (BMS). System działa w dwóch trybach: automatycznym i manualnym. W trybie automatycznym zadajnik analizuje sygnał z położenia kuli zaworu (EV), porównując go z aktualną wartością wyjściową co ustawiony interwał czasowy, na przykład 8 V to odpowiada 80% otwarcia zaworu. Przez użytkownika określany jest interwał czasowy, ustawiony za pomocą encodera z przedziału 5, 15, 30, 45, 60 sekund. Zadajnik dostosowuje napięcie wyjściowe dla pompy, zwiększając lub zmniejszając je o 0,1 V, aby osiągnąć zadany poziom. W przypadku dostępności sygnału z BMS system bezpośrednio przekazując wartość sterującą z BMS do pompy. Tryb manualny pozwala użytkownikowi na bezpośrednie ustawienie napięcia wyjściowego za pomocą encodera, umożliwiając pełną kontrolę nad pracą pompy. Interfejs użytkownika obejmuje wyświetlacz OLED, na którym prezentowane są informacje o aktualnym czasie, trybie pracy, trybie pracy BMS (w przypadku braku wartości wejściowej z BMS użytkownik może zmienić tryb i ustawić priorytetową wartość za pomocą encodera), wartości wejściowych z EV i BMS, napięciu wyjściowym dla pompy oraz aktualnych nastawach. Dodatkowo diody LED sygnalizują aktywność pompy, stan zaworu oraz obecność sygnału z BMS. W przypadku braku sygnału wejściowego z BMS w chwili startu zadajnik startuje z wartości wyjściowej dla pompy równej 10 V.

2.5 Instrukcja obsługi elektronicznego optymalizatora.

Elektroniczny optymalizator jest urządzeniem przeznaczonym do precyzyjnej regulacji pracy pomp w systemach HVAC oraz innych instalacjach wymagających dynamicznego sterowania przepływem medium. Aby rozpocząć pracę, należy podłączyć urządzenie do źródła zasilania 12 V oraz upewnić się, że sygnały z zaworu regulacyjnego EV i systemu BMS są poprawnie podłączone do dedykowanych wejść. Po uruchomieniu urządzenia domyślnie aktywowany jest tryb automatyczny, w którym elektroniczny optymalizator analizuje sygnały napięciowe 0 – 10 V z wejść i dostosowuje napięcie wyjściowe sterujące pompą. Interwał czasowy regulacji można ustawić za pomocą obrotowego encodera, wybierając wartość z przedziału {5, 15, 30, 45, 60} sekund, a wybrany interwał jest prezentowany na wyświetlaczu OLED.

W trybie automatycznym optymalizator priorytetowo wykorzystuje sygnał z systemu BMS, o ile jest dostępny, przekazując bezpośrednio wartość sterującą do pompy. W przypadku braku sygnału z BMS regulacja odbywa się na podstawie pozycji kuli zaworu EV, którą urządzenie porównuje z aktualnym napięciem wyjściowym i koryguje o 0.1 V co zadany interwał czasowy. Istnieje jednak możliwość zmiany źródła wartości

wejściowej w przypadku braku sygnału z BMS. Użytkownik może przełączyć tryb wejściowy na wartość zadaną za pomocą encodera nastawy BMS. W tym trybie encoder pełni funkcję ustawienia priorytetowej wartości, która będzie wykorzystywana do sterowania pompą. Przełączanie trybu źródła sygnału odbywa się za pomocą wbudowanego przycisku encodera.

W trybie manualnym użytkownik może ręcznie ustawić napięcie wyjściowe za pomocą encodera, a jego wartość jest na bieżąco wyświetlana na ekranie OLED. Interfejs użytkownika dostarcza kluczowych informacji o stanie systemu, w tym aktualnego czasu, trybu pracy (automatyczny/manualny), napięć z wejść EV i BMS, napięcia wyjściowego oraz źródła danych wejściowych (BMS_WE lub BMS_ENC).

Dodatkowo diody LED sygnalizują aktywność napięcia wyjściowego, obecność sygnału z BMS oraz sygnału z zaworu EV, co umożliwia szybkie monitorowanie stanu urządzenia. W przypadku braku sygnału z BMS podczas uruchamiania systemu, optymalizator rozpoczyna pracę z maksymalnym napięciem wyjściowym 10 V, co zapewnia pełną moc pompy do czasu ustabilizowania parametrów. Regularne odczyty i przetwarzanie sygnałów są realizowane w sposób ciągły, co gwarantuje płynną pracę urządzenia oraz wysoką precyzję regulacji. W razie problemów, takich jak brak wyświetlania na OLED, nieaktywne diody lub nieprawidłowe działanie encodera, należy sprawdzić poprawność połączeń, wartości sygnałów wejściowych oraz stan techniczny podzespołów.

Rozdział 3

Wykorzystane narzędzia pomiarowe.

Celem niniejszego rozdziału jest przedstawienie narzędzi pomiarowych, które zostały wykorzystane w trakcie realizacji projektu, zarówno podczas pisania oprogramowania dla mikrokontrolera, jak i w procesie badania liniowości zaprojektowanego układu. Opisano tutaj urządzenia, które umożliwiły precyzyjne pomiary kluczowych parametrów systemu, takich jak napięcia, rezystancje oraz inne właściwości elektryczne.

Przeprowadzenie precyzyjnych pomiarów oraz analiza uzyskanych wyników stanowiły kluczowy element weryfikacji poprawności działania zaprojektowanego zadajnika oraz oceny zgodności jego parametrów z założeniami projektowymi. W rozdziale omówiono zasilacz laboratoryjny, który służył jako symulator sygnałów wejściowych, oraz dwa multimetry cyfrowe, wykorzystywane do precyzyjnych pomiarów napięć i rezystancji.

Przedstawienie parametrów technicznych i zakresów działania użytych narzędzi pozwala podkreślić ich rolę w procesie projektowania i testowania układu sterowania, a także zapewnia wiarygodność uzyskanych wyników.

3.1 Zasilacz laboratoryjny.

Zasilacz laboratoryjny DF173005C [13], przedstawiony na rysunku 3.1, to wszechstronne urządzenie przeznaczone do precyzyjnego zasilania układów elektronicznych. Znajduje zastosowanie w laboratoriach, pracach serwisowych oraz procesach prototypowania.

Dokładność napięcia

Dokładność napięcia wyjściowego zasilacza wynosi:

$$\pm(1\% \text{ wartości ustawionej} + 2 \text{ cyfry}),$$



Rysunek 3.1: Zasilacz laboratoryjny DF173005C NDN – 30V, 5A. Zdjęcie ze strony sklepu ndn.com.pl [13].

co gwarantuje stabilne i precyzyjne zasilanie układów wymagających pewnej dokładności [13].

Zakres pracy

Zasilacz umożliwia regulację napięcia w zakresie od 0 V do 30 V, z minimalnym tętnieniem wynoszącym $\leq 1 \text{ mV}_{\text{rms}}$ (dla zakresu częstotliwości od 5 Hz do 1 MHz). Tętnienia i szумы są minimalne, co zapewnia stabilność napięcia wyjściowego, szczególnie w układach czułych na zakłócenia [13].

Zastosowanie zasilacza w projekcie.

Zasilacz DF173005C został wykorzystany w projekcie jako źródło symulowanych sygnałów wejściowych, reprezentujących informacje o położeniu kuli zaworu EV oraz wartości sygnału BMS w zakresie 0 – 10 V oraz dla wejścia sygnału z BMS'u, który też pracuje w tym zakresie.

3.2 Wykorzystane multimetry:

Wykorzystane w projekcie multimetry służyły do pomiarów takich jak: pomiar rezystancji rezystorów, sprawdzenie liniowości dzielnika napięcia dla dobranych rezystorów, badanie liniowości wzmacniacza operacyjnego dla obliczonych parametrów, sprawdzenie liniowości dla wejść układu oraz badanie liniowości wyjścia zadajnika.

3.2.1 Multimetr Sonel

Multimetr cyfrowy Sonel CMM-10 [12], przedstawiony na rysunku 3.2, to zaawansowane narzędzie pomiarowe, umożliwiające precyzyjne pomiary napięcia stałego oraz rezystancji. Jego wszechstronność sprawia, że znajduje zastosowanie w różnorodnych pracach laboratoryjnych i inżynierskich.



Rysunek 3.2: Multimetr Sonel CMM-10.

Multimetr w projekcie był wykorzystywany jako woltomierz do pomiaru wartości napięcia prądu stałego, oraz omomierz - pomiar rezystancji. Poniżej znajduje się opis dokładności miernika dla pomiaru tych dwóch wartości.

Pomiar napięcia stałego (DC)

Multimetr oferuje zakres pomiaru napięcia stałego do 600 V z dokładnością wynoszącą:

$$\pm(0,5\% \text{ wartości mierzonej} + 2 \text{ cyfry}).$$

Rozdzielczość pomiaru wynosi od 0,1 mV, co umożliwia dokładne analizy w szerokim zakresie pomiarowym [12].

Pomiar rezystancji

Urządzenie pozwala na pomiar rezystancji do 40,00 M Ω z dokładnością:

$$\pm(1,0\% \text{ wartości mierzonej} + 2 \text{ cyfry}).$$

Rozdzielczość wynosi od 0,1 Ω , co zapewnia precyzyjne wyniki nawet w przypadku dużych wartości rezystancji [12].

Multimetr NEO.

Multimetr cyfrowy NEO 94-001 [18] pokazany na rysunku 3.3, został wykorzystany w projekcie wyłącznie do pomiarów napięcia prądu stałego. Dlatego poniżej zostały opisane dane dotyczące zakresów, rozdzielczości oraz dokładności, jedynie dla woltomierza prądu stałego.



Rysunek 3.3: Multimetr NEO 94-001.

Zakresy pomiarowe [18]

- 200 mV z rozdzielczością 0,1 mV,
- 2 V z rozdzielczością 0,001 V,
- 20 V z rozdzielczością 0,01 V,
- 200 V z rozdzielczością 0,1 V,

- 600 V z rozdzielczością 1 V.

Dokładność pomiarów [18]

- $\pm(0,8\%$ wartości mierzonej + 5 cyfr) dla zakresów do 200 V,
- $\pm(1,0\%$ wartości mierzonej + 5 cyfr) dla zakresu 600 V.

Impedancja wejściowa

Multimetr charakteryzuje się wysoką impedancją wejściową wynoszącą 10 M Ω , co minimalizuje wpływ miernika na badany obwód [18].

Rozdział 4

Hardware

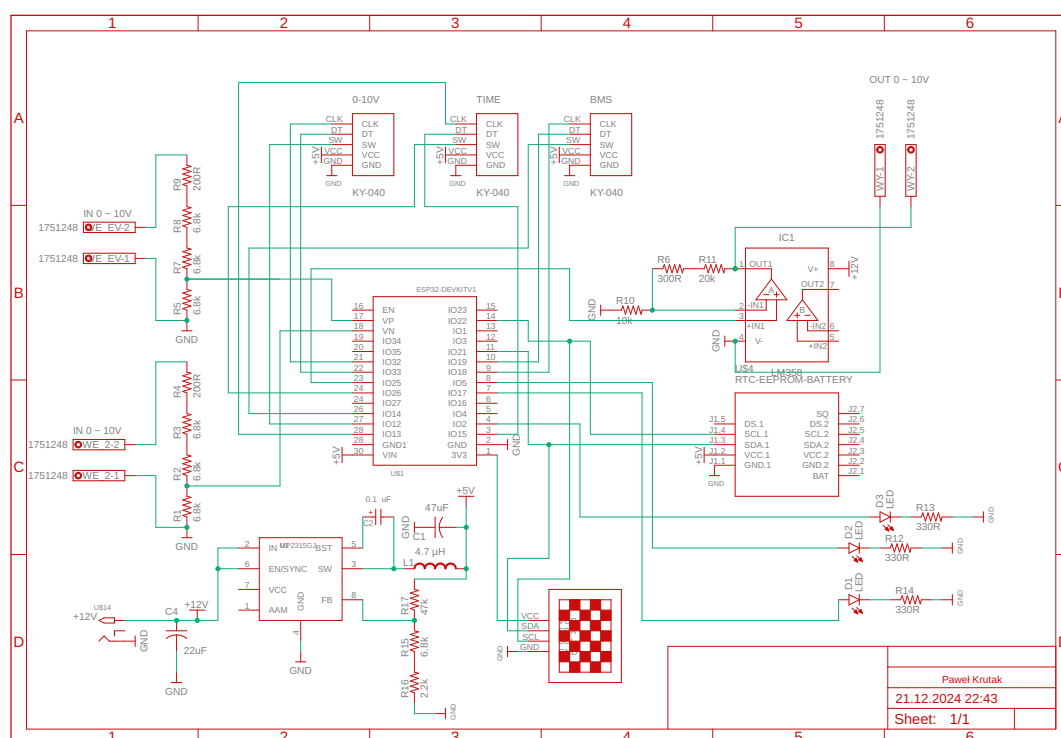
W niniejszym rozdziale przedstawiono szczegółowy opis komponentów sprzętowych wykorzystanych w projekcie, proces projektowania płytki drukowanej (PCB) oraz wykonania obudowy urządzenia. Celem rozdziału jest przedstawienie kluczowych decyzji projektowych, które zadecydowały o funkcjonalności i niezawodności systemu sterowania pompą obiegów.

Omówione zostaną najważniejsze elementy układu, takie jak mikrokontroler ESP32, moduł czasu rzeczywistego RTC DS1307, wzmacniacz operacyjny LM358P oraz przetwornica DC-DC MP2315, wraz z uzasadnieniem ich wyboru oraz opisem ich roli w projekcie. Ponadto, szczególną uwagę poświęcono opisowi układów wspomagających, takich jak rezystory, kondensatory oraz encodery obrotowe, które umożliwiają precyzyjne przetwarzanie sygnałów wejściowych i sterowanie urządzeniami wykonawczymi.

Dalsza część rozdziału skupia się na procesie projektowania płytki PCB, który obejmował rozmieszczenie komponentów, dobór odpowiednich szerokości ścieżek. W ostatniej części rozdziału zaprezentowano projekt obudowy, której konstrukcja została dostosowana do specyfikacji technicznych urządzenia i wykonana z wykorzystaniem technologii druku 3D. Szczegółowe omówienie tych aspektów pozwala na pełne zrozumienie procesu tworzenia sprzętu oraz jego integracji z oprogramowaniem, co jest kluczowe dla osiągnięcia założonej funkcjonalności systemu.

4.1 Schemat połączeń

Przy projektowaniu układu kluczowym krokiem było stworzenie szczegółowego schematu połączeń, który stanowił podstawę do zamówienia potrzebnych elementów, zaprojektowania płytki PCB oraz zaplanowania montażu. Poniższy rysunek 4.1 prezentuje schemat połączeń komponentów zadajnika zaprojektowany w programie EAGLE firmy Autodesk [4].



Rysunek 4.1: Schemat połączeń zadajnika zaprojektowany w programie Autodesk Eagle.

4.2 Opis komponentów.

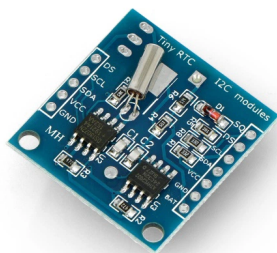
4.2.1 Mikrokontroler ESP32-DevKit v1.



Rysunek 4.2: ESP32 DevKit v1 - zdjęcie ze strony sklepu Botland [9].

ESP32 DevKit v1 przedstawiony na rysunku 4.2 - to mikrokontroler z rodziny esp32 o wysokiej wydajności, wyposażony w zintegrowane moduły Wi-Fi i Bluetooth, co czyni go idealnym do zastosowań IoT. W projekcie pełni rolę centralnej jednostki sterującej, odpowiedzialnej za przetwarzanie danych, komunikację z komponentami peryferyjnymi oraz realizację logiki programu. Komponent zasilany jest napięciem 5V. [25].

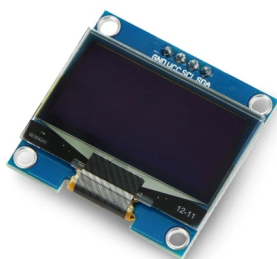
4.2.2 Moduł RTC DS1307 z pamięcią EEPROM 24C32.



Rysunek 4.3: Moduł RTC DS1307 z pamięcią EEPROM 24C32 - zdjęcie ze strony sklepu Botland [20].

Moduł RTC DS1307 - rysunek 4.3 - umożliwia precyzyjne odmierzanie czasu i daty, niezależnie od zasilania, dzięki baterii podtrzymującej. Moduł jest istotny w projekcie aby, móc sterować nastawą czasu. Komunikacja z mikrokontrolerem odbywa się za pomocą interfejsu I²C. Napięcie zasilania tego elementu to 5V [20].

4.2.3 Wyświetlacz OLED 1.3"(I²C).

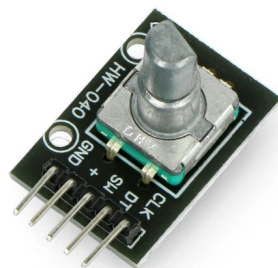


Rysunek 4.4: Wyświetlacz OLED - zdjęcie ze strony sklepu Botland [21].

Wyświetlacz graficzny OLED o przekątnej 1.3 cala i rozdzielczości 128x64 pikseli pokazany na rysunku 4.4 - służy do wizualizacji informacji systemowych wyświetlanych na ekranie OLED. Komunikacja z mikrokontrolerem odbywa się poprzez protokół I²C. Zasilany jest napięciem z zakresu 3.3 – 5 V. [21].

4.2.4 Encoder obrotowy KY-040.

KY-040 to encoder obrotowy, rysunek 4.5, który umożliwia użytkownikowi interakcję z systemem poprzez obrót (ustawienie nastaw) oraz nacisk (zmiana trybów). Pozwala na detekcję kierunku obrotu i precyzyjne zliczanie impulsów [14]. Jest zasilany napięciem 5V.



Rysunek 4.5: Encoder obrotowy KY-040 - zdjęcie ze strony sklepu Botland [7].

4.2.5 Przetwornica DC-DC MP2315

MP2315 to przetwornik typu buck, zdolny do konwersji napięcia wejściowego (12 V) na stabilizowane napięcie wyjściowe (5 V). Jego wysoka sprawność energetyczna pozwala na minimalizację strat mocy i nie duże wydzielanie ciepła, co czyni go odpowiednim do zasilania układów elektronicznych [26].

4.2.6 Wzmacniacz operacyjny LM358P

LM358P to podwójny wzmacniacz operacyjny o niskim poborze mocy, zdolny do pracy z pojedynczym zasilaniem. W projekcie jest wykorzystywany do wzmacniania sygnału analogowego z mikrokontrolera z zakresu 0 – 3.3 V na zakres 0 – 10 V aby ESP32 mógł sterować pompą. [15].

4.2.7 Elementy pasywne

- **Rezystory:** O wartościach od 200 Ω do 47 k Ω , stosowane do ograniczania prądu, podciągania linii sygnałowych oraz ustalania parametrów układów.
- **Kondensatory:** O pojemnościach 0.1 μ F, 22 μ F i 47 μ F, używane do filtrowania napięć i stabilizacji pracy układów.
- **Indukcyjność 4.7 μ H:** Wykorzystywana w obwodzie przetwornicy MP2315 do zapewnienia odpowiedniej filtracji i stabilizacji napięcia wyjściowego.

4.2.8 Diody LED

Diody LED pełnią funkcje sygnalizacyjne, informując o stanie pracy urządzenia, takich jak zasilanie, aktywność modułów czy wystąpienie błędów.

4.3 Kosztorys

Tabela 4.1: Kosztorys komponentów

Nazwa produktu	Koszt [PLN]	Ilość	Link do produktu
ESP32-DEVKITV1	49,90	1	[9]
RTC DS1307 z EEPROM 24C32	5,95	1	[20]
Wyświetlacz OLED 1,3"(niebieski)	29,90	1	[21]
Encoder obrotowy KY-040	7,90	1	[7]
Przetwornica DC-DC MP2315	6,58	1	[26]
Wzmacniacz operacyjny LM358P	7,20	5	[10]
Rezystory i Kondensatory (zestaw)	15	zestaw	
Diody LED	1,90	10	[8]
Przewody	17,50	120	[11]

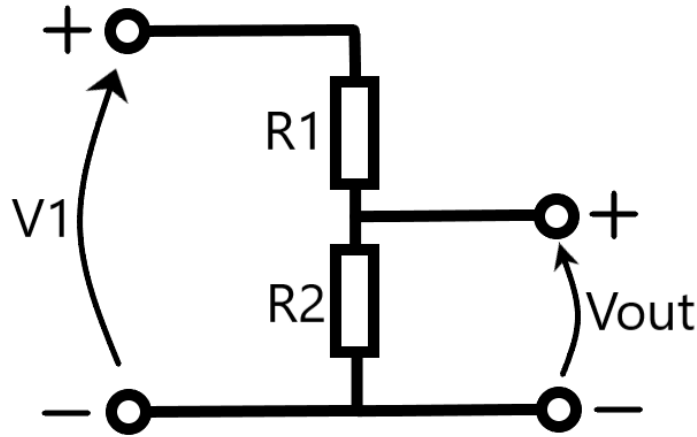
Całkowity koszt komponentów użytych w projekcie, takich jak ESP32-DEVKITV1 (49,90 PLN), RTC DS1307 z EEPROM 24C32 (5,95 PLN), wyświetlacz OLED 1,3" (29,90 PLN), trzy encodery obrotowe KY-040 (łącznie 23,70 PLN), przetwornica DC-DC MP2315 (6,58 PLN), zestaw pięciu wzmacniaczy LM358P (7,20 PLN), rezystory i kondensatory (15,00 PLN), diody LED (1,90 PLN za 10 sztuk) oraz przewody połączeniowe (17,50 PLN), wynosi 157,73 PLN, co zostało szczegółowo przedstawione w tabeli 4.1.

4.4 Wybór Komponentów.

Wybór komponentów został oparty na analizie wymagań projektu.

4.4.1 Zmiana napięcia wejściowego.

Sygnały zewnętrzne, takie jak sygnał z zaworu EV oraz z system zarządzania budynkiem "BMS", dostarczane są w zakresie 0 – 10 V, co jest typowe dla przemysłowych systemów sterowania. Jednakże mikrokontroler ESP32 pracuje na napięciach wejściowe w zakresie 0 – 3.3 V. Dlatego jesteśmy zmuszeni przeskalować tą wartość z zakresu 0 – 10 V na zakres 0 – 3.3 V. Aby rozwiązać ten problem, zastosowano dzielnik napięcia. Schemat dzielnika opisuje rysunek 4.6, który przekształca napięcie wejściowe 0 – 10 V na zakres obsługiwany przez ESP32.



Rysunek 4.6: Dzielnik napięcia.

Wzór na przetwarzanie napięcia w sposób liniowy opisuje wzór - dzielnik napięcia (1) [6]:

$$V_{\text{out}} = V_1 \cdot \frac{R_2}{R_1 + R_2} \quad (1),$$

gdzie:

- V_{out} — napięcie wyjściowe,
- V_1 — napięcie wejściowe,
- R_1 — rezystancja pierwszego rezystora,
- R_2 — rezystancja drugiego rezystora.

Ze względu na brak rezystorów o dokładnych wartościach nominalnych, zastosowano układ zastępczy oparty na sumie szeregowo połączonych rezystancji - opisuje wzór (2) [6]:

$$R_{\text{total}} = R_1 + R_2 + R_3 + \dots + R_n \quad (2),$$

gdzie:

- R_{total} - całkowita rezystancja układu,
- $R_1, R_2, R_3, \dots, R_n$ - rezystancje poszczególnych rezystorów połączonych szeregowo.

Przy napięciu wejściowym $V_1 = 10\text{V}$ oraz rezystorach $R_1 = 13800 \Omega$ i $R_2 = 6800 \Omega$, napięcie wyjściowe według wzoru wyniosło:

$$V_{\text{out}} = \frac{V_1 \cdot R_2}{R_1 + R_2} = \frac{10 \cdot 6800}{13800 + 6800} = 3.30\text{V} \quad (1).$$

Rezystory posiadają pewną swoją tolerancję, która w przypadku użytych elementów wynosiła 5%, co spowodowało minimalne odchylenia od zamierzonych wartości rezystancji w dzielniku napięcia.



(a) Wartość rezystancji rezystora R_1 dzielnika napięcia dla sygnału EV.



(b) Wartość rezystancji rezystora R_2 dzielnika napięcia dla sygnału EV.



(c) Wartość rezystancji rezystora R_1 dzielnika napięcia dla sygnału wejściowego z pompy.



(d) Wartość rezystancji rezystora R_2 dzielnika napięcia dla sygnału wejściowego z pompy.

Rysunek 4.7: Zmierzone wartości rezystancji rezystorów omomierzem.

Te rezystory przedstawione na rysunku 4.7 wykorzystano do analizy liniowości układu wejściowego, którą opisano w rozdziale szóstym, dlatego istotne było udokumentowanie rezystancji tych oporników. Dla zmierzonych wartości rezystancji - uzyskano następujące wyniki:

- Dla sygnału wejściowego z położenia kuli zaworu - EV:

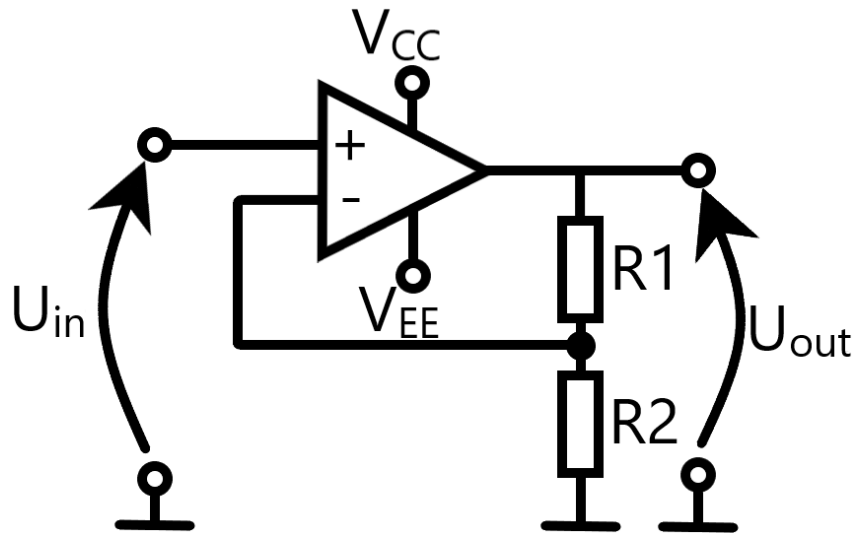
$$V_{\text{out}} = \frac{V_1 \cdot R_2}{R_1 + R_2} = \frac{10 \cdot 6740}{13670 + 6740} = 3.30\text{V} \quad (1).$$

- Dla sygnału wejściowego z pompy - BMS:

$$V_{\text{out}} = \frac{V_1 \cdot R_2}{R_1 + R_2} = \frac{10 \cdot 6790}{13760 + 6790} = 3.30\text{V} \quad (1).$$

4.4.2 Zmiana napięcia wyjściowego.

Aby przekształcić sygnał wyjściowy mikrokontrolera z zakresu 0 – 3.3 V na 0 – 10 V, zastosowano wzmacniacz operacyjny LM358P w konfiguracji nieodwracającej [19]. Tego rodzaju konfiguracja wzmacniacza pozwala na wzmocnienie sygnału wejściowego U_{in} bez odwracania jego fazy, co zapewnia liniowe zwiększenie napięcia w tym zakresie napięciowym.



Rysunek 4.8: Wzmacniacz operacyjny w konfiguracji wzmacniacza nieodwracającego.

Wzmacniacz nieodwracający fazy jest opisany na schemacie - rysunek 4.8.

Zależność między napięciem wejściowym U_{in} , napięciem wyjściowym U_{out} oraz wzmocnieniem k opisana jest równaniem (3) [19]:

$$U_{out} = k \cdot U_{in} = \left(1 + \frac{R_1}{R_2}\right) \cdot U_{in} \quad (3).$$

Współczynnik wzmocnienia k można obliczyć na podstawie wartości rezystorów R_1 oraz R_2 według poniższego wzoru (4) [19]:

$$k = 1 + \frac{R_1}{R_2} \quad (4),$$

gdzie:

- U_{in} — napięcie wejściowe,
- U_{out} — napięcie wyjściowe,
- R_1 — rezystor w gałęzi sprzężenia zwrotnego,
- R_2 — rezystor w gałęzi wejściowej,
- k — współczynnik wzmocnienia wzmacniacza.

Aby uzyskać odpowiednie wzmocnienie $k = \frac{10}{3.3} = 3.03$ (4), wybrano wartości rezystorów $R_1 = 20.3 \text{ k}\Omega$ oraz $R_2 = 10 \text{ k}\Omega$. Takie wartości zapewniają prawidłowe przeskalowanie sygnału, umożliwiając przetworzenie napięcia wejściowego w zakresie $0 - 3.3 \text{ V}$ na napięcie wyjściowe w zakresie $0 - 10 \text{ V}$, co jest wymagane do sterowania pompą.

W celu zapewnienia poprawności wartości użytych rezystorów, zostały one zmierzone za pomocą multimetru, rysunek 4.9. Pomiar rezystancji potwierdził zgodność z wartościami teoretycznymi oporności rezystorów, co jest istotne dla prawidłowego działania układu, ponieważ zakupione rezystory mają tolerancję równą 5%. Te rezystory zostały wykorzystane do badania liniowości układu wyjściowego, które jest opisane w rozdziale szóstym, dlatego istotne było udokumentowanie rezystancji tych oporników.



(a) Wartość rezystancji rezystora R_1



(b) Wartość rezystancji rezystora R_2

Rysunek 4.9: Zmierzone wartości rezystancji rezystorów omomierzem.

Powyższa konfiguracja wzmacniacza operacyjnego, rysunek 4.8, zapewnia stabilne i liniowe wzmocnienie sygnału wejściowego bez odwrócenia jego fazy, co jest kluczowe do precyzyjnej kontroli napięcia wyjściowego zadajnika.

4.4.3 Wybór Przetwornicy Impulsowej do Konwersji Napięcia

W projekcie konwersji napięcia z 12 V na 5 V wybrano przetwornicę impulsową typu *buck* (step-down) **MP2315**. Wybór ten był podyktowany wysoką efektywnością energetyczną, możliwością dostarczenia stabilnego prądu oraz kompaktową konstrukcją, co sprawia, że idealnie nadaje się do implementacji na płytce PCB (lutowanie w technologii SMD) [26].

Alternatywne rozwiązania, takie jak stabilizatory liniowe (np. LM7805), zostały odrzucone z powodu znacznych strat mocy i wynikającego z tego wydzielania ciepła. Przy konwersji z 12 V do 5 V stabilizator liniowy rozpraszałby dużą ilość energii cieplnej, co wymagałoby stosowania radiatorów i zwiększyłoby rozmiar układu. Stabilizator liniowy nie jest również zalecany przy prądach bliskich 1 A , gdyż w takich warunkach obniża sprawność i znacząco zwiększa zapotrzebowanie na chłodzenie.

Rozważano także przypadki z innymi przetwornicami impulsowymi o niższych specyfikacjach, np. **LMR16006**, które zapewniają wyższą sprawność niż stabilizatory liniowe, ale obsługują jedynie prądy do 0,6A. Natomiast przetwornica impulsowa MP2315 może dostarczyć do 3A prądu wyjściowego [26], dlatego zapewnia większy margines bezpieczeństwa i stabilność pracy przy wymaganym obciążeniu 1A, co minimalizuje ryzyko przeciążenia i przedłuża żywotność komponentów.

Obliczenia Rezystorów Dzielnika Napięcia

Do ustawienia napięcia wyjściowego przetwornicy na poziomie 5V zastosowano dzielnik napięcia aby dobrać rezystory do pętli sprzężenia zwrotnego przetwornicy, opierając się na wzorze (5):

$$V_{OUT} = V_{REF} \times \left(1 + \frac{R1}{R2}\right) \quad (5),$$

gdzie:

- V_{OUT} to napięcie wyjściowe ustawione na 5 V,
- V_{REF} to wewnętrzne napięcie referencyjne wynoszące 0,8 V,
- $R1$ i $R2$ to rezystory dzielnika napięcia.

Z przekształcenia wzoru (5) uzyskano stosunek:

$$\frac{R1}{R2} = \frac{5V}{0,8V} - 1 = 4,98 \quad (6).$$

Na podstawie wzoru (6) dobrano standardowe wartości rezystorów:

- $R1 = 47\text{ k}\Omega$,
- $R2 = 9\text{ k}\Omega$.

Dobór Dodatkowych Komponentów

Dla stabilności układu dodano kondensator wejściowy 22 μF (ceramiczny X7R) i kondensator wyjściowy 44 μF . Zastosowano również cewkę o indukcyjności 5,5 μH i prądzie znamionowym 3A, co zachowuje równowagę między wymogami prądowymi, a rozmiarami komponentów.

Dzięki technologii synchronizacji i wysokiej częstotliwości przełączania (500 kHz) przetwornica MP2315 zapewnia wysoką sprawność, minimalizując wydzielanie ciepła i zapewniając stabilne zasilanie przy prądzie wyjściowym do 1A.

Wybrano rezystory o tolerancji 5% ze względu na ich dostępność i wystarczającą dokładność dla założeń projektowych.

4.4.4 Wybór Encoderów Zamiast Potencjometrów do Zadawania Wartości.

W projekcie do zadawania wartości zamiast klasycznych potencjometrów zdecydowano się na zastosowanie encoderów obrotowych. Główną zaletą encoderów jest ich wysoka precyzja oraz możliwość uzyskania większej liczby punktów nastawczych, co umożliwia dokładniejsze sterowanie parametrami układu. Encodery obrotowe, w odróżnieniu od potencjometrów, nie podlegają efektom zużycia styków i są bardziej odporne na zmienne warunki środowiskowe, co zwiększa ich niezawodność oraz żywotność w aplikacjach wymagających wielokrotnego ustawiania wartości.

Dodatkowo, encodery umożliwiają bezpośrednią integrację z cyfrowym interfejsem mikrokontrolera (wyświetlacz OLED), co upraszcza kontrolę i pozwala na cyfrową interpretację zmian pozycji. Encodery oferują również funkcję przycisku, co umożliwia wprowadzenie dodatkowych komend dla użytkownika i jest nieosiągalne w przypadku standardowych potencjometrów.

4.4.5 Wskaźniki LED na Wejściach i Wyjściu Układu

W projekcie zastosowano diody LED, które pełnią funkcję wskaźników obecności napięcia na wejściach i wyjściu układu. Diody te świecą, gdy na odpowiednich pinach podawane jest napięcie, co zapewnia wizualne potwierdzenie działania układu. Dzięki zastosowaniu LED, użytkownik może łatwo monitorować stan zasilania bez konieczności korzystania z dodatkowych przyrządów pomiarowych.

4.4.6 Wyświetlacz do Wizualizacji Parametrów Układu i Czasu

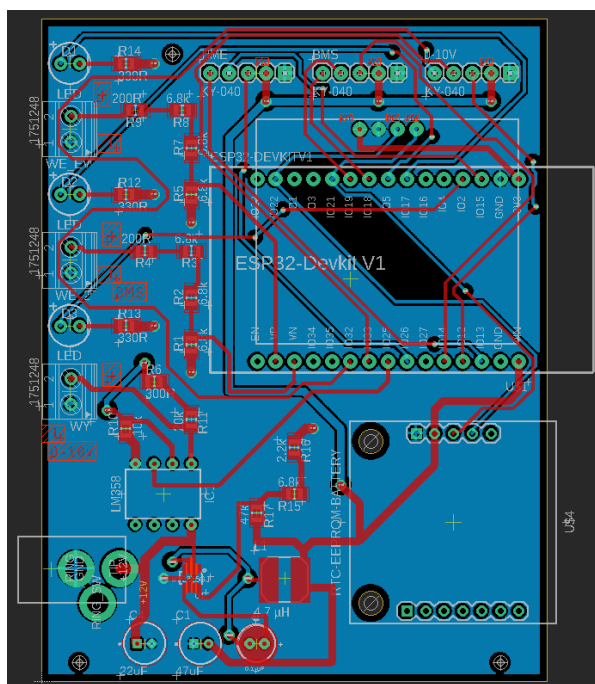
W projekcie zastosowano wyświetlacz OLED o rozdzielczości 128x64 pikseli, który pozwala na monitorowanie kluczowych parametrów pracy układu, takich jak zadana wartość napięcia, aktualne napięcie wyjściowe oraz stan systemu. Dodatkowo, wyświetlacz będzie prezentował bieżący czas, pozyskiwany z modułu RTC (Real Time Clock), co umożliwia śledzenie parametrów w kontekście aktualnego czasu. Dzięki wyświetlaczowi OLED informacje są czytelne i łatwo dostępne dla użytkownika.

4.5 Projektowanie oraz wykonanie płytki drukowanej.

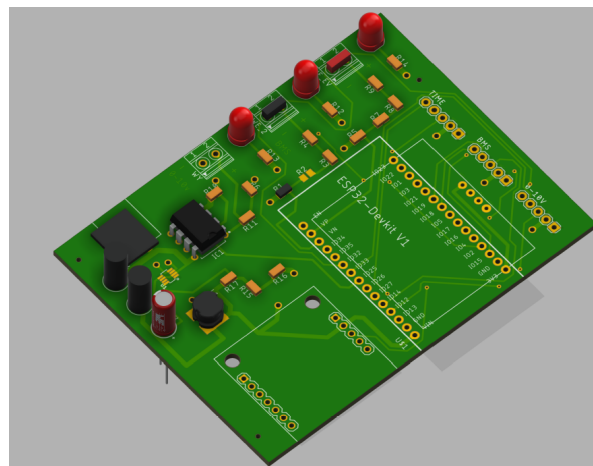
W ramach pracy inżynierskiej wykonano projekt płytki drukowanej (PCB) z wykorzystaniem oprogramowania EAGLE przedstawionej na rysunku 4.10. Proces projektowania obejmował rozmieszczenie elementów na płytce oraz prowadzenie ścieżek

sygnałowych i zasilających zgodnie z założeniami projektowymi. W projekcie zastosowano różne szerokości ścieżek w zależności od poziomów napięć i rodzajów sygnałów:

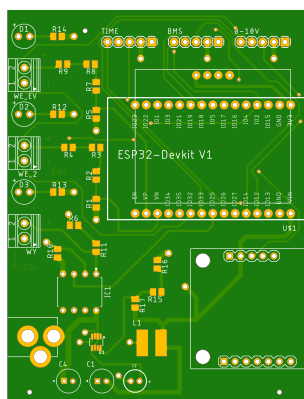
- Ścieżki zasilające 12 V (50 mils) zaprojektowano dla minimalizacji spadków napięcia przy wyższych prądach.
- Dla ścieżek zasilających 5 V przyjęto szerokość 40 mils, co zapewnia optymalne parametry pracy układu.
- Ścieżki dla napięcia 3,3 V posiadają szerokość 30 mils, co jest wystarczające dla układów niskonapięciowych.
- Linie masy (GND) o szerokości 50 mils minimalizują zakłócenia i zapewniają stabilność układu.
- Sygnały analogowe prowadzono ścieżkami o szerokości 20 mils, co umożliwia ograniczenie wpływu zakłóceń elektromagnetycznych.
- Sygnały cyfrowe zaprojektowano z szerokością ścieżek 15 mils, co jest odpowiednie dla poprawnego przesyłu danych.



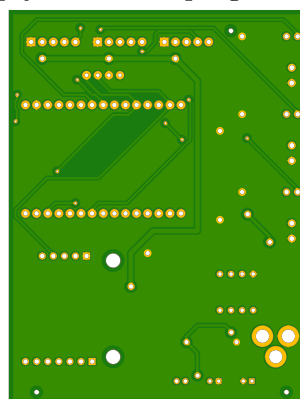
(a) Zaprojektowana płytki PCB w programie EAGLE.



(b) Model płytki PCB w programie FUSION.



(c) Wygenerowany obraz górnej warstwy w programie EAGLE.



(d) Wygenerowany obraz dolnej warstwy w programie EAGLE.

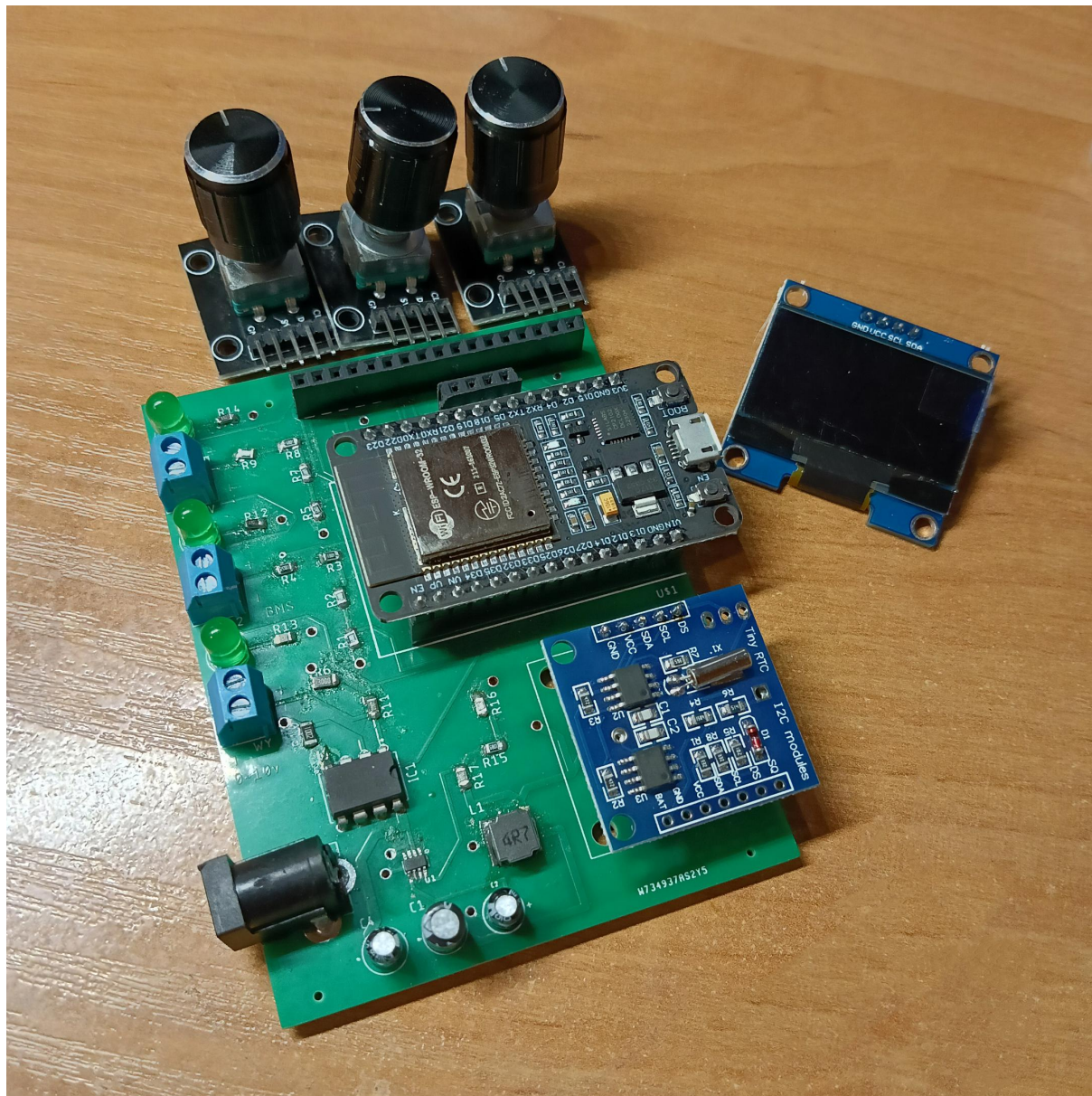
Rysunek 4.10: Model zaprojektowanej płytki PCB.

Na dolnej warstwie zaprojektowano wylaną miedź pełniącą funkcję wspólnej masy (GND). Takie rozwiązanie zapewnia lepsze właściwości elektryczne, redukuje impedancję masy oraz dodatkowo minimalizuje zakłócenia elektromagnetyczne w układzie.

W trakcie prac projektowych szczególnie przydatna okazała się dokumentacja oprogramowania EAGLE [4], która dostarczyła istotnych informacji na temat projektowania układów PCB, takich jak zasady tworzenia wylewek masy, konfiguracja szerokości ścieżek oraz eksport plików Gerber do produkcji. Dzięki tym wskazówkom możliwe było optymalne zaprojektowanie płytki PCB.

Ostateczny projekt płytki został wyeksportowany w formacie Gerber i wydrukowany przez firmę PCBWay, co zapewniło wysoką jakość wykonania oraz zgodność z założeniami projektowymi. Po dostarczeniu zamówionych komponentów oraz

wydrukowanej płytki PCB przystąpiono do procesu lutowania elementów w technologii SMD i THT, co umożliwiło zmontowanie i uruchomienie układu zgodnie z założeniami projektowymi. Do lutowania wykorzystano lutownicę HotAir oraz lutownicę oporową. Końcowy wygląd płytki drukowanej znajduje się na rysunku 4.11.

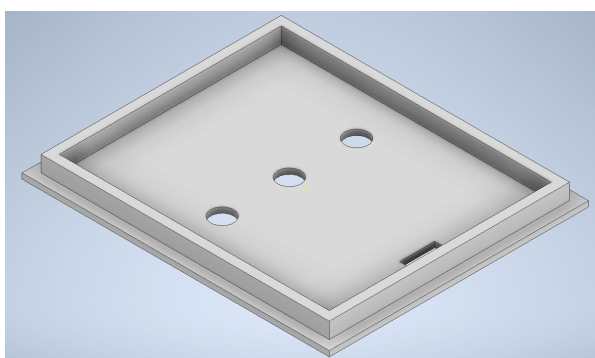


Rysunek 4.11: Płytką drukowaną z przylutowanymi komponentami.

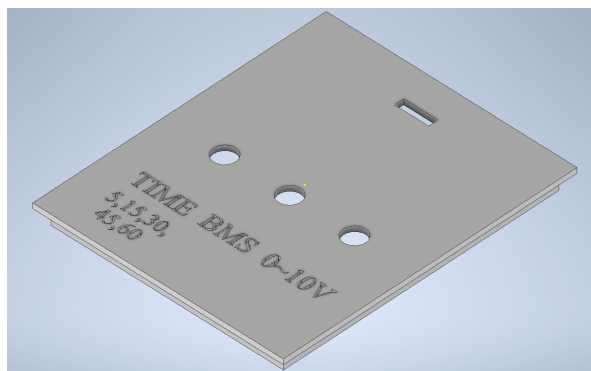
4.6 Projektowanie i wydruk obudowy.

Proces projektowania i wytwarzania obudowy dla zasilacza obejmował kilka kluczowych etapów, poczynając od stworzenia modelu 3D w programie Autodesk Inventor [3], a kończąc na wydruku elementów przy użyciu drukarki 3D.

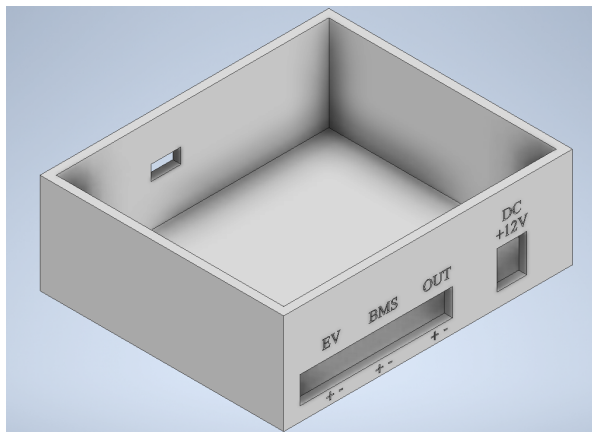
Autodesk Inventor jest jednym z najbardziej zaawansowanych narzędzi do projektowania CAD (Computer-Aided Design), pozwalającym na tworzenie precyzyjnych modeli 3D oraz symulacje mechaniczne [3]. W ramach projektu zaprojektowano obudowę składającą się z dwóch głównych części - górnej i dolnej. Obudowa została zaprojektowana z uwzględnieniem wymiarów komponentów elektronicznych oraz otworów dla zasilania wejść oraz wyjścia zasilacza. Zaprojektowana obudowa znajduje się na rysunku 5.27.



(a) Zaprojektowana obudowa górna w programie Inventor - widok z góry.



(b) Zaprojektowana obudowa górna w programie Inventor - widok z dołu.



(c) Zaprojektowana obudowa dolna w programie Inventor.

Rysunek 4.12: Model zaprojektowanej obudowy w programie Inventor.

Gotowy model 3D został wyeksportowany w formacie STL, co pozwoliło na jego obróbkę w slicerze kompatybilnym z drukarką 3D Bambu Lab A1 [17]. Drukarka ta jest wyposażona w zaawansowane funkcje, takie jak automatyczna kalibracja stołu oraz obsługa wielomateriałowych wydruków, co gwarantuje wysoką precyzję i jakość

wykonanych elementów.

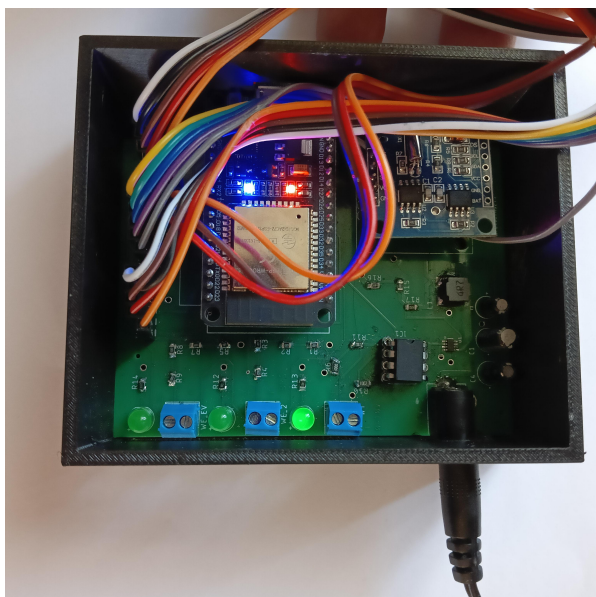
Do wydruku obudowy wykorzystano filament PET-G [2], który charakteryzuje się:

- wysoką odpornością na temperaturę i promieniowanie UV,
- dobrą wytrzymałością mechaniczną,
- niską absorpcją wilgoci,
- łatwością przetwarzania na drukarkach 3D.

Proces drukowania został podzielony na dwa etapy: wydruk dolnej i górnej części obudowy. Każda część obudowy została zaprojektowana tak, aby precyzyjnie dopasować górną część do dolnej bez potrzeby stosowania dodatkowych elementów mocujących.

4.7 Końcowy wygląd elektronicznego optymalizatora.

Prezentacja końcowego wyglądu zadajnika ukazuje pełną integrację komponentów elektronicznych wraz z obudową, co stanowi kluczowy etap realizacji założeń projektowych. Na rysunku 4.13 zaprezentowano ostateczny wygląd zadajnika po procesach lutowania oraz wydrukowania obudowy.



(a) Wygląd zadajnika w środku obudowy.



(b) Wygląd zadajnika - widok na wejścia i wyjście układu.



(c) Wygląd zadajnika - widok z góry.

Rysunek 4.13: Końcowy wygląd zaprojektowanego zadajnika.

Rozdział 5

Software

W niniejszym rozdziale przedstawiono implementację kodu zadajnika zaprojektowanego w celu realizacji funkcji związanych z automatycznym sterowaniem pompą. Zadajnik został opracowany na mikrokontrolerze ESP32, co umożliwia integrację różnych peryferiów oraz realizację złożonych algorytmów przetwarzania danych. Opracowany system charakteryzuje się modułowością oraz elastycznością, co pozwala na jego adaptację w różnych warunkach pracy.

Główne funkcje zadajnika obejmują regulację pracy pompy w trybie automatycznym i manualnym, z uwzględnieniem odczytów z systemu zarządzania budynkiem (BMS) oraz informacji o położeniu kuli zaworu. System umożliwia również obsługę wejść analogowych i cyfrowych, w tym precyzyjny odczyt napięć z wejść oraz obsługę encoderów używanych do ustawiania parametrów pracy. Komunikacja z urządzeniami peryferyjnymi odbywa się za pomocą magistrali I²C, co obejmuje moduł czasu rzeczywistego (RTC) oraz wyświetlacz OLED, umożliwiający wizualizację danych procesowych. Ponadto zadajnik wykorzystuje zaawansowane algorytmy przetwarzania sygnałów pomiarowych, takie jak interpolacja, korekta offsetu oraz wygładzanie eksponencjalne, co zapewnia stabilność i precyzję działania.

Ostateczne sterowanie wyjściowe realizowane jest poprzez generowanie sygnałów PWM oraz zarządzanie sygnałami analogowymi i cyfrowymi, które sterują urządzeniami wykonawczymi, takimi jak pompa czy wyświetlacz. W dalszej części rozdziału szczegółowo omówiono wykorzystane biblioteki, przypisanie funkcji do poszczególnych pinów mikrokontrolera, algorytmy przetwarzania danych oraz logikę regulacji w obu trybach pracy. Przedstawienie tych elementów ma na celu przybliżenie zarówno aspektów technicznych, jak i uzasadnienia zastosowanych rozwiązań, co pozwala na lepsze zrozumienie funkcjonowania zadajnika w kontekście realizowanego projektu.

5.1 Kod zadajnika.

5.1.1 Opis wykorzystanych bibliotek.

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <U8g2lib.h>
4 #include <RTClib.h>
5 #include <esp_adc_cal.h>
```

Rysunek 5.1: Wykorzystane biblioteki.

Na rysunku 5.1 znajduje się fragment kodu do zaimportowania pięciu bibliotek, które są niezbędne do realizacji funkcjonalności programu:

- **Biblioteka `Arduino.h`** [27]

Biblioteka ta jest integralną częścią środowiska programistycznego Arduino i zapewnia dostęp do podstawowych funkcji oraz definicji umożliwiających komunikację z mikrokontrolerem. Przykładowe funkcje, które udostępnia, to:

- `digitalRead()` i `digitalWrite()` do odczytu i zapisu stanów na pinach cyfrowych,
- `analogRead()` i `analogWrite()` do obsługi wejść i wyjść analogowych,
- `millis()` do pomiaru czasu od uruchomienia urządzenia.

Biblioteka `Arduino.h` jest fundamentem, na którym budowane są wszystkie bardziej zaawansowane opcje programu.

- **Biblioteka `Wire.h`** [28]

`Wire.h` umożliwia komunikację przez magistralę I²C (Inter-Integrated Circuit), co jest szczególnie istotne w przypadku urządzeń peryferyjnych takich jak moduły RTC czy wyświetlacze OLED. Umożliwia ona inicjalizację magistrali oraz przesyłanie danych pomiędzy urządzeniami w prosty sposób. W projekcie magistrala I²C jest wykorzystywana do komunikacji z:

- zegarem czasu rzeczywistego (RTC),
- wyświetlaczem OLED.

- **Biblioteka `U8g2lib.h`** [16]

Biblioteka ta służy do obsługi wyświetlaczy graficznych OLED. Umożliwia ona wyświetlanie tekstu, kształtów graficznych oraz wybór czcionek o różnych rozmiarach. W projekcie wyświetlacz OLED pełni kluczową rolę w prezentacji informacji o stanie systemu w czasie rzeczywistym, takich jak bieżący czas,

aktualne wartości nastaw, odczytanych wartości wejściowych, wiadomość o wartości wyjściowej oraz aktualny tryb pracy.

- **Biblioteka `RTCLib.h`** [1]

`RTCLib.h` wspiera ona komunikację z zegarami czasu rzeczywistego (RTC). Moduły RTC umożliwiają synchronizację systemu z rzeczywistym czasem kalendarzowym, co jest kluczowe w aplikacjach wymagających precyzyjnego ustalania harmonogramowania działań, np. regulacji napięcia w określonych przedziałach czasowych. Biblioteka ta oferuje funkcje takie jak:

- inicjalizacja modułu RTC,
- odczyt bieżącej daty i czasu,
- automatyczna aktualizacja czasu podczas resetu mikrokontrolera.

- **Biblioteka `esp_adc_cal.h`** [23]

Biblioteka kalibracyjna dla wbudowanego przetwornika analogowo-cyfrowego (ADC) w mikrokontrolerze ESP32. ESP32 posiada funkcje pomiaru napięcia na pinach wejściowych ADC, jednak ze względu na specyfikę sprzętową pomiary mogą być obciążone błędami. Biblioteka `esp_adc_cal.h` pozwala na:

- interpolację wyników odczytu ADC,
- zastosowanie kalibracji napięć referencyjnych,
- poprawę dokładności pomiarów napięciowych.

W projekcie biblioteka ta jest wykorzystywana do precyzyjnego odczytu napięć z położenia F EV i BMS.

5.1.2 Uzasadnienie zastosowania bibliotek.

Zastosowanie powyższych bibliotek umożliwia realizację złożonych zadań w prosty i modularny sposób. Użycie gotowych rozwiązań pozwala na redukcję błędów programistycznych oraz skrócenie czasu implementacji. Każda z zaimportowanych bibliotek pełni unikalną rolę, która wspiera zarówno obsługę sprzętową, jak i implementację funkcjonalności logicznych.

5.1.3 Opis zdefiniowanych pinów

Rysunek 5.2 przedstawia fragment kodu do definicji pinów dla mikrokontrolera, które odpowiadają za różne funkcje systemu. Zostały one przypisane do określonych elementów sprzętowych zgodnie ze schematem projektu. Poniżej znajduje się szczegółowy opis:

```
1 #define CLK_BMS 18
2 #define DT_BMS 19
3 #define CLK_VOLT 32
4 #define DT_VOLT 33
5 #define CLK_TIME 13
6 #define DT_TIME 15
7 #define BUTTON_PIN 26
8 #define ENCODER_BUTTON_PIN 14
9 #define ANALOG_OUT 25
10 #define VP_PIN 36
11 #define VN_PIN 39
12 #define DIGITAL_OUT 2
13 #define DIGITAL_BMS 5
14 #define DIGITAL_EV 17
```

Rysunek 5.2: Definicja pinów mikrokontrolera.

- **CLK_BMS (pin 18)**
Pin wejściowy zegarowy dla encodera od nastawy BMS.
- **DT_BMS (pin 19)**
Pin odpowiedzialny za odczyt danych (*Data*) z encodera związanego z funkcjonalnością BMS. Wraz z sygnałem zegarowym umożliwia określenie kierunku i liczby impulsów encodera.
- **CLK_VOLT (pin 32)**
Służy jako wejście zegarowe dla encodera, od nastawy wartości wyjściowej w trybie manualnym. Działa w połączeniu z pinem DT_VOLT.
- **DT_VOLT (pin 33)**
Odpowiada za odczyt danych z encodera sterującego napięciem.
- **CLK_TIME (pin 13)**
Pin wejściowy zegarowy dla encodera, który służy do ustawiania interwałów czasowych regulacji o wartościach 5s, 15s, 30s, 45s oraz 60s.
- **DT_TIME (pin 15)**
Pin do odczytu danych z encodera obsługującego ustawienia czasowe. W połączeniu z CLK_TIME umożliwia zmianę interwałów czasu regulacji.
- **BUTTON_PIN (pin 26)**
Pin odpowiadający za obsługę przycisku encodera. Umożliwia ręczne przełączanie trybów pracy systemu.
- **ENCODER_BUTTON_PIN (pin 14)**
Pin obsługujący przycisk wbudowany w encoder. Służy do zmiany źródła wartości

wejściowych pomiędzy BMS_INPUT a BMS_ENCODER, co pozwala na przełączanie trybu pracy w zależności od dostępności sygnału z systemu BMS.

- **ANALOG_OUT (pin 25)**

Wyjście analogowe, które steruje sygnałem napięciowym w zakresie od 0 V do 3.3 V. Wykorzystywane do sterowania pompą.

- **VP_PIN (pin 36)**

Pin ADC (Analog-to-Digital Converter), który jest używany do odczytu napięcia z położenia kuli zaworu EV. Napięcie to informuje o bieżącym położeniu zaworu.

- **VN_PIN (pin 39)**

Pin ADC 3, odpowiedzialny za odczyt napięcia z BMS'u. Napięcie to informuje o docelowej wartości silnika pompy i jest informacją priorytetową.

- **DIGITAL_OUT (pin 2)**

Wyjście cyfrowe sterujące diodą sygnalizacyjną. Służy do wskazywania, czy napięcie wyjściowe (sterujące zaworem) jest aktywne (+0.1 V).

- **DIGITAL_BMS (pin 5)**

Pin cyfrowy sterujący diodą sygnalizującą aktywność systemu BMS. Dioda zapala się, gdy napięcie na pinie VN przekracza minimalny próg (0.1 V).

- **DIGITAL_EV (pin 17)**

Pin cyfrowy odpowiedzialny za sygnalizację aktywności systemu zaworu. Dioda podłączona do tego pinu świeci, gdy napięcie pinu VP przekracza minimalny próg (0.1 V).

5.1.4 Konfiguracja wyświetlacza OLED

```
1 U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/
    U8X8_PIN_NONE);
```

Rysunek 5.3: Definicja wyświetlacza OLED.

Na rysunku 5.3 definiujemy obiekt `u8g2`, który jest instancją klasy `U8G2_SH1106_128X64_NONAME_F_HW_I2C` z biblioteki `U8g2lib`. Obiekt ten służy do obsługi wyświetlacza OLED o rozdzielczości 128x64 pikseli, korzystającego ze zadajnika SH1106 i sprzętowego interfejsu I²C [16].

Parametry obiektu

- U8G2_R0 — Parametr określający orientację wyświetlacza jako domyślną (bez rotacji).
- U8X8_PIN_NONE — Wskazuje, że pin resetu nie jest używany. Reset wyświetlacza realizowany jest sprzętowo lub w sposób wewnętrzny przez bibliotekę.

5.1.5 Konfiguracja modułu RTC.

```
1 RTC_DS1307 rtc;
```

Rysunek 5.4: Definicja RTC.

Na rysunku 5.4 przedstawiono fragment kodu odpowiedzialny za definicję obiektu `rtc`, który jest instancją klasy `RTC_DS1307` z biblioteki `RTClib` [1]. Obiekt ten reprezentuje moduł zegara czasu rzeczywistego (RTC) i odpowiada za synchronizację systemu z aktualnym czasem kalendarzowym.

Zachowanie w przypadku błędu

Jeżeli komunikacja z modułem RTC nie zostanie nawiązana, program wygeneruje ostrzeżenie o błędzie, co umożliwi użytkownikowi szybką identyfikację problemu.

5.1.6 Definicje trybów pracy i źródeł wartości

```
1 enum Mode { AUTOMATIC, MANUAL };  
2 Mode currentMode = AUTOMATIC; // Domyślny tryb  
3  
4 enum ValueSource { BMS_INPUT, BMS_ENCODER };  
5 ValueSource currentValueSource = BMS_INPUT; // Domyślny tryb
```

Rysunek 5.5: Typ wyliczeniowy ENUM.

Opis trybów pracy

Rysunek 5.5 pokazuje dwie możliwości działania systemu, które zostały zdefiniowane za pomocą wyliczeniowego typu danych `enum Mode`:

- **AUTOMATIC** - Tryb automatyczny, w którym zadajnik reguluje napięcie wyjściowe na podstawie sygnałów z czujników i zaimplementowanej logiki sterowania.

- **MANUAL** - Tryb ręczny, w którym użytkownik może samodzielnie ustawiać napięcie wyjściowe przy pomocy encodera.

Domyślny tryb systemu ustawiono jako **AUTOMATIC**, co pozwala na rozpoczęcie pracy zadajnika z pełną funkcjonalnością automatycznej regulacji.

Opis źródeł wartości wejściowych

Zmienna wyliczeniowa `ValueSource` definiuje dwa możliwe źródła wartości wejściowych:

- **BMS_INPUT** - Wartości wejściowe są pobierane z encodera jako nastawa BMS. Ten tryb jest stosowany, gdy nie posiadamy sygnału wejściowego z BMS.
- **BMS_ENCODER** - Wartości wejściowe są ustawiane ręcznie za pomocą encodera. Tryb ten jest aktywowany, gdy sygnał z BMS jest niedostępny i użytkownik chce wymusić priorytet.

Domyślnym źródłem wartości wejściowych jest **BMS_INPUT**, co odzwierciedla priorytet, na sygnał wejściowy z BMS.

5.1.7 Funkcja korekty offsetu napięcia.

```

1 // Korekta offsetu dla dolnych, średnich i górnych wartości
2 float applyOffsetCorrection(float voltage) {
3
4     if (voltage > 0.0 && voltage <= 2.7) {
5         return voltage + 0.1; // Korekta dla dolnego zakresu
6     }
7     else if (voltage > 2.7 && voltage <= 3.0) {
8         return voltage - 0.05; // Korekta dla górnego zakresu
9     }
10    else if (voltage > 3.0 && voltage <= 3.05) {
11        return voltage - 0.1; // Korekta dla górnego zakresu
12    }
13    else if (voltage > 3.05 && voltage <= 3.1) {
14        return voltage - 0.2; // Korekta dla górnego zakresu
15    }
16    else if (voltage > 3.1 && voltage <= 3.14){
17        return voltage - 0.3; // Korekta dla górnego zakresu
18    }
19    return voltage; // Bez korekcji poza zakresem
20 }
```

Rysunek 5.6: Korekta offsetu napięcia dla wejść zadajnika.

Opis funkcji.

Funkcja `applyOffsetCorrection()` została przedstawiona na rysunku 5.6, jest odpowiedzialna za wprowadzenie korekty offsetu dla wartości napięcia wejściowego w zależności od jego zakresu. Korekta ta pozwala na precyzyjne dopasowanie sygnału pomiarowego do oczekiwanego zakresu pracy, eliminując błędy wynikające z tolerancji sprzętowej.

Zasada działania.

Funkcja przyjmuje pojedynczy parametr `voltage` typu `float` reprezentujący napięcie wejściowe, a następnie:

- Sprawdza, do którego zakresu należy wartość napięcia.
- Wprowadza odpowiednią korektę offsetu w zależności od przedziału.
- Zwraca skorygowaną wartość napięcia.

Znaczenie dla projektu.

Korekta offsetu jest istotna w celu poprawy niedoskonałości sprzętowych, takich jak:

- Błędy pomiarowe przetwornika analogowo-cyfrowego (ADC),
- Tolerancje sprzętu, np. różnice w czujnikach napięcia,
- Nieliniowości w sygnałach wejściowych.

Dzięki zastosowaniu tej funkcji możliwe jest uzyskanie bardziej stabilnych i precyzyjnych wyników pomiarowych, co przekłada się na lepszą jakość regulacji w systemie sterowania pompą.

5.1.8 Funkcja interpolacji kawałkowej.

Funkcję `interpolateVoltage()` pokazana na rysunku 5.7 realizuje interpolację kawałkową, przeliczając wartość odczytaną z przetwornika analogowo-cyfrowego (ADC) [23] na odpowiadające jej napięcie woltowe. Wartości graniczne (`ADC_POINTS`) oraz odpowiadające im napięcia referencyjne (`VREFS`) definiują przedziały, w których funkcja zwraca wynik.

Opis działania.

- **Przeszukiwanie przedziałów:** Pętla `for` analizuje kolejne przedziały zdefiniowane przez `ADC_POINTS`. Jeżeli wartość `adc_reading` mieści się w danym przedziale, to następuje obliczenie współczynnika skalowania (`slope`).

```

1 // Funkcja interpolacji kawałkowej
2 float interpolateVoltage(uint32_t adc_reading) {
3     for (int i = 0; i < 3; i++) {
4         if (adc_reading >= ADC_POINTS[i] && adc_reading <= ADC_POINTS
5             [i + 1]) {
6             // Oblicz współczynnik skalowania w bieżącym zakresie
7             float slope = (VREFS[i + 1] - VREFS[i]) / (ADC_POINTS[i +
8                 1] - ADC_POINTS[i]);
9             return VREFS[i] + slope * (adc_reading - ADC_POINTS[i]);
10        }
11    }
12    // Zabezpieczenie przed wartościami poza zakresem
13    return adc_reading < ADC_POINTS[0] ? VREFS[0] : VREFS[3];
14 }

```

Rysunek 5.7: Funkcja interpolacji kawałkowej.

- **Obliczanie napięcia:** Interpolacja jest realizowana według wzoru (7):

$$\text{result} = \text{VREFS}[i] + \text{slope} \cdot (\text{adc_reading} - \text{ADC_POINTS}[i]) \quad (7),$$

gdzie *slope* to nachylenie linii interpolacyjnej w bieżącym przedziale.

- **Zabezpieczenie przed wartościami poza zakresem:** Jeśli *adc_reading* znajduje się poza zakresem określonym przez *ADC_POINTS*, zwracana jest wartość skrajna z *VREFS*.

Znaczenie dla projektu

Interpolacja kawałkowa pozwala na precyzyjne przeliczenie wartości z ADC na napięcie woltowe, eliminując nieliniowości i niedokładności przetwornika. Dzięki temu funkcja umożliwia dokładne odwzorowanie sygnałów analogowych w zakresie zdefiniowanym przez użytkownika, co jest kluczowe dla poprawnego działania systemu sterowania w projekcie.

5.1.9 Funkcja wygładzania eksponencjalnego.

```

1 float applyExponentialSmoothing(float currentValue, float
2     previousValue, float alpha) {
3     return alpha * currentValue + (1 - alpha) * previousValue;
4 }

```

Rysunek 5.8: Funkcja wygładzania eksponencjalnego.

Opis funkcji

Funkcja `applyExponentialSmoothing()`, którą przedstawia rysunek 5.8, realizuje wygładzanie eksponencjalne, które pozwala na eliminację szumu z danych pomiarowych. Wygładzanie eksponencjalne nadaje większą wagę nowszym wartościom, jednocześnie zachowując część informacji z poprzednich pomiarów. Funkcja ta jest powszechnie stosowana w przetwarzaniu sygnałów i danych pomiarowych.

Parametry wejściowe

- `currentValue` - Aktualna wartość pomiaru (`float`), - napięcie odczytane z wejścia.
- `previousValue` - Poprzednia wygładzona wartość (`float`), przechowywana w systemie z poprzedniego kroku obliczeń.
- `alpha` — Współczynnik wygładzania (`float`), którego wartość mieści się w zakresie od 0 do 1. Określa on stopień wagi nadawanej nowym pomiarom:
 - Dla wartości `alpha` bliskiej 1 większy nacisk kładzie się na bieżące dane, co prowadzi do szybszej reakcji na zmiany.
 - Dla wartości `alpha` bliskiej 0 większą wagę mają dane historyczne, co prowadzi do bardziej stabilnego, ale wolniejszego dostosowywania.

Działanie funkcji

Funkcja oblicza wygładzoną wartość na podstawie poniższego równania (8):

$$\text{Wynik} = \alpha \cdot \text{currentValue} + (1 - \alpha) \cdot \text{previousValue} \quad (8),$$

- Składnik $\alpha \cdot \text{currentValue}$ odpowiada za wprowadzenie bieżących danych do wyniku.
- Składnik $(1 - \alpha) \cdot \text{previousValue}$ uwzględnia wpływ poprzednich wartości.

Znaczenie dla projektu

W projekcie elektronicznego optymalizatora pompy funkcja `applyExponentialSmoothing()` odgrywa ważną rolę w przetwarzaniu danych z czujników, takich jak:

- Wygładzanie pomiarów napięcia (położenia kuli zaworu EV (pin VP) i BMS (pin VN)), które mogą być zaszumione z powodu niedoskonałości przetwornika ADC lub dokładności przetwarzania napięcia z zakresu 0 – 10 V na 0 – 3.3 V.
- Poprawa stabilności algorytmów sterowania poprzez redukcję wahań w sygnałach wejściowych.

Zalety stosowania wygładzania eksponencjalnego

- Prosta implementacja i niski koszt obliczeniowy.
- Efektywne tłumienie szumów przy zachowaniu reaktywności na zmiany w sygnałach wejściowych.
- Możliwość dostosowania do różnych aplikacji dzięki regulowanemu parametrowi α .

Funkcja ta jest kluczowym elementem systemu, zapewniającym dokładne i stabilne dane do dalszego przetwarzania.

5.1.10 Funkcja zmiany trybu pracy systemu.

```
1 // Funkcja zmiany trybu
2 void changeMode() {
3     currentMode = (currentMode == AUTOMATIC) ? MANUAL : AUTOMATIC;
4 }
```

Rysunek 5.9: Funkcja zmiany trybu pracy systemu.

Opis funkcji

Funkcja `changeMode()` znajdująca się na rysunku 5.9, odpowiada za przełączanie trybu pracy systemu pomiędzy trybem automatycznym (`AUTOMATIC`) a ręcznym (`MANUAL`). Realizuje to w prosty sposób za pomocą operatora warunkowego (ternary operator).

5.1.11 Funkcja obsługi encodera BMS.

```

1 // Funkcja obsługi encodera BMS
2 void IRAM_ATTR handleEncoderBMS() {
3     int currentStateCLK = digitalRead(CLK_BMS);
4     if (currentStateCLK != lastStateCLK_BMS && currentStateCLK ==
        HIGH) {
5         if (digitalRead(DT_BMS) != currentStateCLK) {
6             if (counter_bms < 100) counter_bms++;
7         } else {
8             if (counter_bms > 0) counter_bms--;
9         }
10        bms_value = counter_bms * 0.1;
11    }
12    lastStateCLK_BMS = currentStateCLK;

```

Rysunek 5.10: Funkcja obsługi encodera BMS.

Funkcja `handleEncoderBMS()` przedstawiona na rysunku 5.10 obsługuje sygnały z encodera odpowiedzialnego za sterowanie systemem BMS. W przerwaniu sprzętowym (`IRAM_ATTR`) odczytywany jest aktualny stan pinu zegarowego (`CLK_BMS`). Przy każdej zmianie stanu na `HIGH`, funkcja analizuje kierunek obrotu encodera, porównując stan pinu danych (`DT_BMS`) z bieżącym stanem zegara. Wartość licznika `counter_bms` jest odpowiednio zwiększana lub zmniejszana w zakresie od 0 do 100. Obliczona wartość `bms_value`, jako iloczyn `counter_bms` i współczynnika 0.1, odzwierciedla ustawienie encodera w jednostkach napięcia dla zakresu 0 – 10 V. Funkcja umożliwia płynną regulację parametrów wejściowych systemu BMS w czasie rzeczywistym.

5.1.12 Funkcja obsługi encodera VOLT.

```

1 void IRAM_ATTR handleEncoderVOLT() {
2     int currentStateCLK = digitalRead(CLK_VOLT);
3     if (currentStateCLK != lastStateCLK_VOLT && currentStateCLK ==
        HIGH) {
4         if (digitalRead(DT_VOLT) != currentStateCLK) {
5             if (counter_volt < 100) counter_volt++;
6         } else {
7             if (counter_volt > 0) counter_volt--;
8         }
9         volt_value = counter_volt * 0.1;
10    }
11    lastStateCLK_VOLT = currentStateCLK;
12 }

```

Rysunek 5.11: Funkcja obsługi encodera VOLT.

Funkcja `handleEncoderVOLT()`, która pokazuje rysunek 5.11, działa w sposób analogiczny do funkcji `handleEncoderBMS`, jednak jej zadaniem jest obsługa encodera odpowiedzialnego za ustawienie napięcia wyjściowego w zakresie 0 – 10 V. Funkcja jest używana w trybie `MANUAL`, umożliwiając użytkownikowi ręczną regulację napięcia wyjściowego zadajnika.

5.1.13 Funkcja obsługi encodera TIME.

```

1 int values_time[] = {5, 15, 30, 45, 60}; // Wartości czasowe dla
   enodera TIME
2
3 // Funkcja obsługi encodera TIME
4 void IRAM_ATTR handleEncoderTIME() {
5     int currentStateCLK = digitalRead(CLK_TIME);
6     if (currentStateCLK != lastStateCLK_TIME && currentStateCLK ==
       HIGH) {
7         if (digitalRead(DT_TIME) != currentStateCLK) {
8             if (index_time < (sizeof(values_time) / sizeof(values_time
               [0]) - 1)) {
9                 index_time++;
10            }
11        } else {
12            if (index_time > 0) {
13                index_time--;
14            }
15        }
16    }
17    lastStateCLK_TIME = currentStateCLK;
18 }

```

Rysunek 5.12: Funkcja obsługi encodera TIME.

Funkcja `handleEncoderTIME()`, którą prezentuje rysunek 5.12, obsługuje encoder do nastawy jednego z predefiniowanych interwałów czasowych zapisanych w tablicy `values_time`, która zawiera wartości: {5, 15, 30, 45, 60} sekund. Funkcja działa w przerwaniu sprzętowym (`IRAM_ATTR`), co zapewnia szybkie reagowanie na sygnały z encodera. Odczytywany jest stan pinu zegarowego `CLK_TIME`. W momencie zmiany stanu na `HIGH` analizowany jest kierunek obrotu encodera na podstawie stanu pinu `DT_TIME`.

Jeśli encoder obraca się w jednym kierunku, indeks `index_time` jest zwiększany o 1 (do maksymalnej wartości odpowiadającej ostatniemu elementowi tablicy). W przeciwnym kierunku `index_time` jest zmniejszany o 1 (do minimalnej wartości 0). Wybrany interwał czasowy odpowiada wartości w tablicy `values_time[index_time]`, co umożliwia dynamiczne ustawienie czasu regulacji w trybie automatycznym. Wybrany

czas można zapisać wzorem (9):

$$\text{time_interval} = \text{values_time}[\text{index_time}] \quad (9),$$

gdzie `time_interval` to aktualny czas w sekundach, zależny od pozycji encodera.

5.1.14 Inicjalizacja interfejsów komunikacyjnych i urządzeń peryferyjnych

```
1 Serial.begin(115200);  
2 Wire.begin(21, 22);  
3 u8g2.begin();
```

Rysunek 5.13: Inicjalizacja komunikacji.

Na rysunku 5.13 zamieszczono fragment kodu odpowiadający za uruchomienie podstawowych interfejsów komunikacyjnych i urządzeń peryferyjnych. Funkcja `Serial.begin(115200)` inicjalizuje port szeregowy UART z prędkością 115200 bitów na sekundę, co umożliwia komunikację z komputerem w celach diagnostycznych i debugowania. Funkcja `Wire.begin(21, 22)` [28] inicjalizuje magistralę I²C, przypisując piny 21 (SDA) i 22 (SCL) do komunikacji z urządzeniami peryferyjnymi, takimi jak moduł RTC i wyświetlacz OLED. Funkcja `u8g2.begin()` uruchamia bibliotekę `U8g2lib` [16], umożliwiając obsługę wyświetlacza OLED.

5.1.15 Sprawdzenie obecności modułu RTC.

```
1 if (!rtc.begin()) {  
2   Serial.println("RTC nie znaleziono!");  
3   while (1);  
4 }
```

Rysunek 5.14: Sprawdzenie poprawności działania modułu RTC.

Powyższy fragment kodu przedstawiony na rysunku 5.14 weryfikuje, czy moduł zegara czasu rzeczywistego (RTC) został poprawnie zainicjalizowany. Funkcja `rtc.begin()` [1] próbuje uruchomić komunikację z modułem RTC. Jeśli moduł nie zostanie wykryty, na porcie szeregowym wyświetlany jest komunikat diagnostyczny "RTC nie znaleziono!", a program wchodzi w nieskończoną pętlę `while(1)`. Taka konstrukcja pozwala zatrzymać działanie systemu w przypadku krytycznego błędu, umożliwiając użytkownikowi zidentyfikowanie problemu i podjęcie działań naprawczych.

5.1.16 Synchronizacja czasu modułu RTC.

```

1 if (!rtc.isrunning()) {
2   rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
3 }

```

Rysunek 5.15: Synchronizacja czasu modułu RTC.

Rysunek 5.15 prezentuje fragment kodu, który sprawdza, czy moduł zegara czasu rzeczywistego (RTC) jest aktywny i posiada ustawiony czas. Funkcja `rtc.isrunning()` [1] weryfikuje, czy moduł RTC działa poprawnie. Jeśli czas nie został wcześniej ustawiony (np. po pierwszym uruchomieniu lub wymianie baterii podtrzymującej), funkcja `rtc.adjust()` synchronizuje zegar modułu z czasem kompilacji programu. W tym celu wykorzystywane są makra `__DATE__` i `__TIME__`, które przechowują datę i godzinę kompilacji kodu [1].

Ten mechanizm zapewnia, że moduł RTC zawsze posiada poprawne dane czasowe, co jest kluczowe dla wyświetlania aktualnego czasu oraz realizacji zadań związanych z synchronizacją w systemie, takich jak interwały regulacji w trybie automatycznym.

5.1.17 Inicjalizacja wyświetlacza OLED i komunikat powitalny.

```

1 u8g2.clearBuffer();
2 u8g2.setFont(u8g2_font_ncenB08_tr);
3 u8g2.drawStr(0, 10, "Start programu!");
4 u8g2.sendBuffer();
5 delay(2000);

```

Rysunek 5.16: Inicjalizacja wyświetlacza OLED

Powyższy fragment kodu pokazany na rysunku 5.16, odpowiada za przygotowanie wyświetlacza OLED do pracy oraz wyświetlenie komunikatu powitalnego. Funkcja `u8g2.clearBuffer()` [16] czyści bufor wyświetlacza, umożliwiając rozpoczęcie renderowania nowej zawartości. Następnie funkcja `u8g2.setFont(u8g2_font_ncenB08_tr)` [16] ustawia czcionkę `ncenB08_tr`, która jest stylizowaną czcionką o rozmiarze 8 punktów. Tekst "Start programu!" jest rysowany na ekranie przy pomocy funkcji `u8g2.drawStr()` [16], zaczynając od pozycji (0, 10) na ekranie. Funkcja `u8g2.sendBuffer()` [16] przesyła zawartość bufora do wyświetlacza, dzięki czemu tekst staje się widoczny, a `delay(2000)` wprowadza 2-sekundowe opóźnienie, umożliwiając użytkownikowi zapoznanie się z komunikatem. Fragment ten pełni funkcję informacyjną, sygnalizując użytkownikowi poprawne uruchomienie programu.

5.1.18 Konfiguracja trybów pracy pinów mikrokontrolera.

```

1 pinMode(CLK_BMS, INPUT_PULLUP);
2 pinMode(DT_BMS, INPUT_PULLUP);
3 pinMode(CLK_VOLT, INPUT_PULLUP);
4 pinMode(DT_VOLT, INPUT_PULLUP);
5 pinMode(CLK_TIME, INPUT_PULLUP);
6 pinMode(DT_TIME, INPUT_PULLUP);
7 pinMode(BUTTON_PIN, INPUT_PULLUP);
8 pinMode(ANALOG_OUT, OUTPUT);
9 pinMode(ENCODER_BUTTON_PIN, INPUT_PULLUP);
10 pinMode(DIGITAL_OUT, OUTPUT);
11 pinMode(DIGITAL_BMS, OUTPUT);
12 pinMode(DIGITAL_EV, OUTPUT);

```

Rysunek 5.17: Konfiguracja trybów pracy pinów mikrokontrolera.

Powyższy fragment kodu przedstawiony na rysunku 5.17, konfiguruje tryby pracy pinów mikrokontrolera ESP32 przy użyciu funkcji `pinMode()` [27], dostosowując je do wymagań projektu. Piny wejściowe (`INPUT_PULLUP`) obsługują sygnały zegarowe (`CLK_BMS`, `CLK_VOLT`, `CLK_TIME`), dane z encoderów (`DT_BMS`, `DT_VOLT`, `DT_TIME`), oraz przyciski (`BUTTON_PIN`, `ENCODER_BUTTON_PIN`), wykorzystując wewnętrzne rezystory podciągające. Piny wyjściowe (`OUTPUT`) sterują sygnałami dla urządzeń peryferyjnych, takich jak pompa (`ANALOG_OUT`) i diody sygnalizacyjne (`DIGITAL_OUT`, `DIGITAL_BMS`, `DIGITAL_EV`). Taka konfiguracja zapewnia poprawne odczyty danych wejściowych oraz umożliwia generowanie sygnałów sterujących, co jest kluczowe dla prawidłowej współpracy mikrokontrolera z urządzeniami peryferyjnymi i elementem wykonawczym w projekcie [25].

5.1.19 Inicjalizacja stanów początkowych encoderów.

```

1 lastStateCLK_BMS = digitalRead(CLK_BMS);
2 lastStateCLK_VOLT = digitalRead(CLK_VOLT);
3 lastStateCLK_TIME = digitalRead(CLK_TIME);

```

Rysunek 5.18: Inicjalizacja stanów początkowych encoderów.

Rysunek 5.18 prezentuje fragment kodu, który inicjalizuje początkowe stany sygnałów zegarowych dla encoderów używanych w projekcie. Każda zmienna (`lastStateCLK_BMS`, `lastStateCLK_VOLT`, `lastStateCLK_TIME`) przechowuje wartość odczytaną z odpowiedniego pinu wejściowego (`CLK_BMS`, `CLK_VOLT`, `CLK_TIME`) za pomocą funkcji `digitalRead()` [27]. Operacja ta pozwala na zapamiętanie aktualnego stanu sygnału zegarowego, co jest niezbędne do poprawnego określania kierunku i liczby

impulsów obrotowych encodera w dalszej części programu. Dzięki tej inicjalizacji system ma punkt odniesienia dla analizy zmian sygnału zegarowego, co umożliwia precyzyjne sterowanie różnymi funkcjami, takimi jak ustawienia BMS, napięcia wyjściowego i interwałów czasowych.

5.1.20 Konfiguracja przerwań dla encoderów.

```

1 attachInterrupt(digitalPinToInterrupt(CLK_BMS), handleEncoderBMS,
  CHANGE);
2 attachInterrupt(digitalPinToInterrupt(CLK_VOLT),
  handleEncoderVOLT, CHANGE);
3 attachInterrupt(digitalPinToInterrupt(CLK_TIME),
  handleEncoderTIME, CHANGE);

```

Rysunek 5.19: Konfiguracja przerwań dla encoderów.

Rysunek 5.19 opisuje fragment kodu, który konfiguruje przerwania dla pinów zegarowych encoderów, co pozwala na obsługę sygnałów w czasie rzeczywistym. Funkcja `attachInterrupt()` [27] przypisuje przerwanie do każdego pinu wejściowego (`CLK_BMS`, `CLK_VOLT`, `CLK_TIME`), wskazując funkcję obsługującą odpowiedni (`handleEncoderBMS`, `handleEncoderVOLT`, `handleEncoderTIME`) oraz typ zdarzenia `CHANGE`, które aktywuje przerwanie przy każdej zmianie stanu sygnału (z `HIGH` na `LOW` lub odwrotnie). Dzięki temu encodery mogą być obsługiwane niezależnie od głównej pętli programu, co zapewnia szybką i precyzyjną reakcję systemu na zmiany sygnałów. Konfiguracja przerwań jest kluczowa dla precyzyjnego sterowania parametrami BMS, napięciem wyjściowym oraz interwałami czasowymi.

5.1.21 Inicjalizacja początkowego stanu wyjściowego pompy.

```

1 // Początkowy stan (100% mocy pompy)
2 if (firstRun) {
3     outputVoltage = 10.0; // Ustawienie na maksymalną moc
4     firstRun = false;
5 }

```

Rysunek 5.20: Inicjalizacja początkowego stanu wyjściowego pompy

Powyższy fragment kodu zamieszczony na rysunku 5.20 ustawia początkowy stan napięcia wyjściowego sterującego pompą na maksymalną wartość 10.0 V, co odpowiada pracy pompy na pełnej mocy. Mechanizm ten jest kontrolowany za pomocą flagi logicznej `firstRun`, która przy pierwszym uruchomieniu programu ma wartość `true`. Po ustawieniu maksymalnego napięcia, zmienna `firstRun` jest ustawiana na `false`, aby

zapobiec wielokrotnemu wykonywaniu tego bloku kodu. Inicjalizacja początkowego stanu zapewnia, że pompa rozpoczyna pracę z pełną mocą, a następnie przechodzi do normalnego sterowania zgodnie z dalszą logiką programu.

5.1.22 Obsługa przycisku z eliminacją drgań styków (*debouncing*).

```

1 // Obsługa przycisku z eliminacją drgań styków
2 bool reading = digitalRead(BUTTON_PIN);
3 if (reading != lastButtonState1) {
4     lastDebounceTime1 = millis();
5 }
6 if ((millis() - lastDebounceTime1) > debounceDelay1) {
7     if (reading != buttonState1) {
8         buttonState1 = reading;
9         if (buttonState1 == LOW) { // Aktywacja na zboczu opadają
10             cym
11                 changeMode();
12         }
13     }
14     lastButtonState1 = reading;

```

Rysunek 5.21: Obsługa przycisku z eliminacją drgań styków (*debouncing*).

Powyższy fragment kodu przedstawiony na rysunku 5.21, obsługuje przycisk z mechanizmem eliminacji drgań styków (*debouncing*). Drgania styków to naturalne zakłócenia występujące podczas mechanicznego wciskania lub zwalniania przycisków, które powodują wielokrotne, krótkotrwałe zmiany sygnału. Mechanizm *debouncing* wprowadza opóźnienie czasowe, dzięki któremu system rejestruje jedynie stabilne zmiany stanu przycisku. Po odczytaniu bieżącego stanu przycisku (`digitalRead(BUTTON_PIN)`) [27], kod sprawdza, czy stan różni się od poprzedniego (`lastButtonState1`). W przypadku zmiany, aktualizowany jest czas zmiany stanu za pomocą `millis()` [27], a następnie, jeśli od ostatniej zmiany minęło więcej niż `debounceDelay1`, stan przycisku jest aktualizowany. Jeśli nowy stan to `LOW` (zbocze opadające), wywoływana jest funkcja `changeMode()`, zmieniająca tryb pracy systemu. Mechanizm ten zapewnia stabilność działania przycisku, eliminując fałszywe odczyty, co jest kluczowe w systemach automatyki wymagających niezawodnej obsługi wejść.

5.1.23 Odczyt, przetwarzanie i skalowanie napięć z pinów: VP i VN.

```

1 // Odczyt wartości VP i VN
2 uint32_t adcVP = analogRead(VP_PIN);
3 uint32_t adcVN = analogRead(VN_PIN);
4
5 // Interpolacja i korekta napięć
6 float vpVoltage = interpolateVoltage(adcVP)
7   );
7 float vnVoltage = interpolateVoltage(adcVN)
8   );
9
10 // Wygładzanie sygnałów
11 filteredVP = applyExponentialSmoothing(vpVoltage, filteredVP,
12   ALPHA);
11 filteredVN = applyExponentialSmoothing(vnVoltage, filteredVN,
12   ALPHA);
13
14 // Skalowanie napięć do 0-10 V
15 float scaledVP = filteredVP * (10.0 / 3.3);
16 float scaledVN = filteredVN * (10.0 / 3.3);

```

Rysunek 5.22: Odczyt, przetwarzanie i skalowanie napięć odczytanych z pinów: VP i VN.

Powyższy fragment kodu pokazany na rysunku 5.22, realizuje odczyt, przetwarzanie i skalowanie wartości napięć z pinów analogowych VP_PIN i VN_PIN. Funkcja `analogRead()` [27] odczytuje surowe dane z przetwornika analogowo-cyfrowego (ADC), które następnie są przetwarzane przez `interpolateVoltage()` w celu przeliczenia na wartości napięcia oraz przez `applyOffsetCorrection()` dla korekcji offsetu. Przetworzone napięcia są poddawane wygładzaniu eksponencjalnemu przy użyciu `applyExponentialSmoothing()`, co pozwala na redukcję szumów. Ostatecznie, wygładzone wartości są skalowane z zakresu 0 – 3.3 V na 0 – 10 V poprzez przemnożenie przez współczynnik $\frac{10.0}{3.3}$. Przetworzone i przeskalowane wartości (`scaledVP` i `scaledVN`) zapewniają dokładne i stabilne dane wejściowe dla dalszych obliczeń w zadajniku.

5.1.24 Logika regulacji w trybie automatycznym.

Fragment kodu, który znajduje się na rysunku 5.23, implementuje logikę regulacji w trybie automatycznym, w którym napięcie wyjściowe `outputVoltage` jest sterowane na podstawie odczytów z różnych źródeł danych. Interwał regulacji (`adjustmentInterval`) jest ustawiany w zależności od wartości wybranej za pomocą encodera. Jeśli od ostatniej regulacji upłynął czas dłuższy niż ustawiony interwał, system analizuje dane wejściowe. W pierwszej kolejności uwzględniany jest sygnał z BMS (`scaledVN`), jeśli jego wartość

przekracza 0 i tryb pracy to BMS_INPUT. W przypadku braku takiego sygnału, priorytet ma wartość z encodera (bms_value) w trybie BMS_ENCODER. Jeżeli żaden z tych warunków nie jest spełniony to regulacja odbywa się na podstawie informacji o położeniu kuli zaworu EV (scaledVP) w odniesieniu do outputVoltage. Następnie wartość napięcia jest przeliczana na sygnał PWM i przesyłana do wyjścia ANALOG_OUT za pomocą analogWrite() [27], co pozwala na sterowanie urządzeniami wykonawczymi w zakresie 0–10V.

```

1 // Ustawienie czasu regulacji na podstawie encodera
2 int adjustmentInterval = values_time[index_time];
3
4 // Logika regulacji w trybie automatycznym
5 if (currentMode == AUTOMATIC) {
6     unsigned long currentMillis = millis();
7     if (currentMillis - lastAdjustmentTime >= adjustmentInterval
8         * 1000) { //Sprawdzenie interwału czasu
9         //Priorytet sygnału BMS
10        if (scaledVN >= 0.1 && currentValueSource == BMS_INPUT) {
11            outputVoltage = scaledVN; // Korzystaj z wartości
12            BMS_INPUT, jeśli VN > 0
13        } else if (bms_value >= 0.1 && currentValueSource ==
14            BMS_ENCODER) {
15            outputVoltage = bms_value; // W przeciwnym razie użyj
16            encodera
17        } else {
18            if (scaledVP < outputVoltage) { //Jeśli położenie
19            zaworu mniejsze niż zadane
20            outputVoltage = max(outputVoltage - 0.1, 0.0); //
21            Zmniejsz napięcie
22        } else if (scaledVP > outputVoltage) { //Jeśli położ
23        enie zaworu większe niż zadane
24            outputVoltage = min(outputVoltage + 0.1, 10.0);
25            //Zwiększ napięcie
26        } else {
27            outputVoltage = scaledVP;
28        }
29        lastAdjustmentTime = currentMillis; //Aktualizacja czasu
30        regulacji
31    }
32    //Aktualizacja PWM w trybie automatycznym
33    int pwmValue1 = (outputVoltage / 10.0) * 255;
34    analogWrite(ANALOG_OUT, pwmValue1);
35 }

```

Rysunek 5.23: Logika regulacji w trybie automatycznym.

5.1.25 Logika regulacji w trybie ręcznym.

```

1 if (currentMode == MANUAL) {
2     int pwmValue = (volt_value / 10.0) * 255;
3     analogWrite(ANALOG_OUT, pwmValue);
4 }

```

Rysunek 5.24: Logika regulacji w trybie ręcznym.

Rysunek 5.24 obejmuje fragment kodu, który implementuje logikę sterowania w trybie ręcznym (MANUAL). Gdy system znajduje się w tym trybie, wartość napięcia wyjściowego (`volt_value`) jest bezpośrednio przeliczana na sygnał PWM w zakresie 0–255 przy pomocy równania (10):

$$\text{pwmValue} = \left(\frac{\text{volt_value}}{10.0} \right) \cdot 255 \quad (10),$$

gdzie `volt_value` reprezentuje napięcie wyjściowe w zakresie od 0 V do 10 V. Obliczona wartość PWM jest następnie przesyłana na wyjście `ANALOG_OUT` przy użyciu funkcji `analogWrite()` [27]. Logika ta pozwala użytkownikowi ręcznie kontrolować napięcie wyjściowe za pomocą encodera, bez interwencji automatycznego systemu regulacji, co zapewnia elastyczność w dostosowywaniu parametrów pracy do indywidualnych potrzeb.

5.1.26 Obsługa diod sygnalizacyjnych.

```

1 digitalWrite(DIGITAL_OUT, (outputVoltage > 0.1) ? HIGH : LOW);
2 digitalWrite(DIGITAL_BMS, (scaledVN > 0.1) ? HIGH : LOW);
3 digitalWrite(DIGITAL_EV, (scaledVP > 0.1) ? HIGH : LOW);

```

Rysunek 5.25: Obsługa diod sygnalizacyjnych.

Rysunek 5.25 przedstawia fragment kodu do sterowania diodami sygnalizacyjnymi, które wizualizują stan kluczowych parametrów systemu. Dioda `DIGITAL_OUT` wskazuje aktywność wyjścia zadajnika - jest włączana (`HIGH`), gdy wartość napięcia `outputVoltage` przekracza 0.1 V, co sygnalizuje, że zadajnik dostarcza napięcie. Dioda `DIGITAL_BMS` informuje o obecności sygnału z systemu BMS, włączając się, gdy `scaledVN` jest większe niż 0.1 V. Dioda `DIGITAL_EV` sygnalizuje aktywność sygnału z zaworu, włączając się, gdy wartość `scaledVP` przekracza 0.1 V. Mechanizm ten zapewnia użytkownikowi intuicyjną informację o stanie wejść i wyjścia co ułatwia diagnozowanie i monitorowanie pracy urządzenia w czasie rzeczywistym.

5.1.27 Wyświetlanie informacji na wyświetlaczu OLED.

```

1 // Wyświetlanie na OLED
2 DateTime now = rtc.now();
3 u8g2.clearBuffer();
4 u8g2.setCursor(0, 10);
5 u8g2.printf("Czas: %02d:%02d:%02d", now.hour(), now.minute(), now
    .second());
6 u8g2.setCursor(0, 20);
7 u8g2.printf("Tryb: %s", currentMode == AUTOMATIC ? "AUTO" : "
    MANUAL");
8 u8g2.setCursor(0, 30);
9 u8g2.printf("EV: %.1fV BMS: %.1fV", scaledVP, scaledVN);
10 // Wyświetlanie wartości wyjściowej w zależności od trybu
11 if (currentMode == AUTOMATIC) {
12     u8g2.setCursor(0, 40);
13     u8g2.printf("Output: %.1fV", outputVoltage);
14 } else if (currentMode == MANUAL) {
15     u8g2.setCursor(0, 40);
16     u8g2.printf("Output: %.1fV", volt_value);
17 }
18 u8g2.setCursor(0, 50);
19 u8g2.printf("BMS_IN: %s", currentValueSource == BMS_INPUT ? "
    BMS_INPUT" : "BMS_ENC");
20 u8g2.setCursor(0, 60);
21 u8g2.printf("N: T=%ds B=%.1fV V=%.1fV", adjustmentInterval,
    bms_value, volt_value);
22 u8g2.sendBuffer();

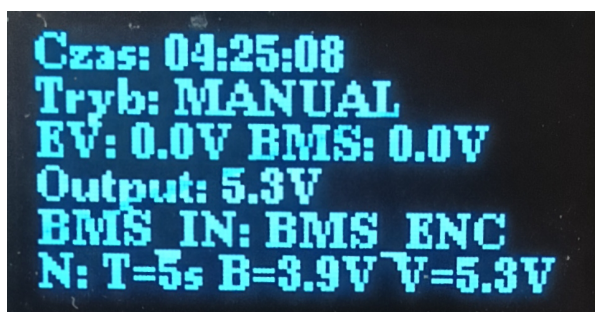
```

Rysunek 5.26: Wyświetlanie informacji na wyświetlaczu OLED.

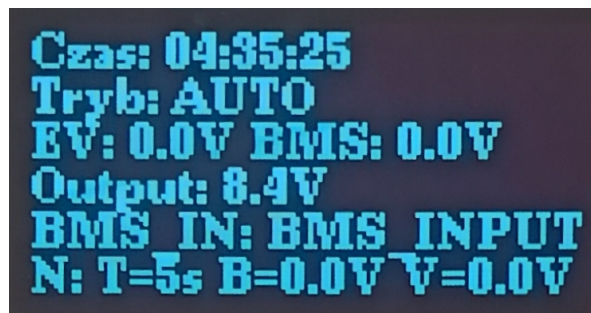
Powyższy fragment kodu pokazany na rysunku 5.26 odpowiada za wyświetlanie kluczowych informacji o stanie systemu na wyświetlaczu OLED, które możemy zobaczyć na rysunku 5.27b. Wykorzystuje obiekt `u8g2` z biblioteki `U8g2lib` [16], który pozwala na renderowanie tekstu na buforze graficznym, a następnie przesyła zawartości do wyświetlacza za pomocą `u8g2.sendBuffer()` [16].

Opis działania

- **Czas rzeczywisty:** Funkcja `rtc.now()` pobiera aktualny czas z modułu RTC, a `u8g2.printf()` [16] wyświetla go w formacie `hh:mm:ss` na pierwszej linii.
- **Tryb pracy:** Na drugiej linii wyświetlany jest aktualny tryb pracy systemu, który zależy od zmiennej `currentMode` (AUTO lub MANUAL).
- **Napięcia:** Na trzeciej linii wyświetlane są napięcia `scaledVP` i `scaledVN` reprezentujące wartości odczytane z wejść.



(a) Wyświetlanie na wyświetlaczu OLED w trybie MANUAL.



(b) Wyświetlanie na wyświetlaczu OLED w trybie AUTO.

Rysunek 5.27: Wyświetlanie informacji na wyświetlaczu OLED.

- **Wyjście:** Na czwartej linii wyświetlane jest napięcie wyjściowe `outputVoltage` w trybie automatycznym lub `volt_value` w trybie ręcznym.
- **Źródło BMS:** Na piątej linii wskazane jest aktualne źródło danych BMS (`BMS_INPUT` lub `BMS_ENC`).
- **Parametry:** Ostatnia linia wyświetla parametry regulacji, takie jak interwał czasowy (`adjustmentInterval`) oraz napięcia `bms_value` i `volt_value`.

5.1.28 Realizacja operacji w regularnych odstępach czasu.

Rysunek 5.28 opisuje fragment kodu, który implementuje mechanizm umożliwiający wykonywanie operacji w regularnych odstępach czasu bez blokowania pętli programu. Funkcja `millis()` [27] zwraca czas w milisekundach od uruchomienia mikrokontrolera, a zmienna `interval` definiuje żądany odstęp czasowy. Warunek `if (currentMillis - previousMillis >= interval)` sprawdza, czy od ostatniego wykonania operacji upłynął czas równy lub dłuższy niż `interval`. Po spełnieniu tego warunku następuje aktualizacja zmiennej `previousMillis`, zapisującej czas ostatniego wykonania kodu, co umożliwia cykliczne wykonywanie zaplanowanych działań.

```

1 unsigned long currentMillis = millis();
2 if (currentMillis - previousMillis >= interval) {
3     previousMillis = currentMillis;
4     // Kod do wykonania w regularnych odstępach czasu
5 }

```

Rysunek 5.28: Realizacja operacji w regularnych odstępach czasu.

Ten mechanizm jest lepszą alternatywą niż funkcja `delay()` [27], ponieważ te procesy mogą działać równolegle. Oraz jest kluczowy w systemach czasu rzeczywistego, takich jak zadajniki automatyki, gdzie istotne jest wykonywanie zadań w regularnych

odstępach czasu, np. aktualizacja wyświetlacza OLED, monitorowanie danych wejściowych czy sterowanie urządzeniami peryferyjnymi.

5.2 Podsumowanie.

Implementacja oprogramowania zapewnia modułowość, elastyczność oraz wysoką precyzję regulacji pracy pompy, co spełnia założenia projektowe i umożliwia dalsze rozwijanie systemu.

Rozdział 6

Weryfikacja i walidacja

6.1 Weryfikacja i walidacja

Celem niniejszego rozdziału jest przedstawienie wyników weryfikacji i walidacji układu elektronicznego optymalizatora, które mają na celu potwierdzenie poprawności działania systemu w warunkach rzeczywistych. Proces weryfikacji obejmował szczegółową analizę kluczowych parametrów, takich jak liniowość przetwarzania sygnałów wejściowych i wyjściowego, dokładność pomiarów oraz zgodność wyników z założeniami projektowymi.

Szczególną uwagę poświęcono badaniom liniowości przetwarzania sygnałów przez dzielniki napięć, wzmacniacz operacyjny oraz przetwornik ADC mikrokontrolera ESP32. Przeprowadzono również pomiary napięć referencyjnych i rzeczywistych dla sygnałów wejściowych, a także sygnału wyjściowego, aby dokładnie sprawdzić poprawność działania zaprojektowanego układu.

Przedstawione w tym rozdziale wyniki oraz przeprowadzone analizy umożliwiają ocenę, czy układ spełnia założone wymagania projektowe, a także identyfikację obszarów, które mogą wymagać dalszej pracy.

6.2 Badanie liniowości przetwarzanego napięcia z wejść i wyjścia elektronicznego optymalizatora.

Podczas wyboru komponentów zadajnika kluczowym elementem było zapewnienie liniowości przetwarzania napięć zarówno na wejściu, jak i na wyjściu układu. Liniowość ta jest niezbędna dla zachowania dokładności działania systemu sterowania, ponieważ wszelkie nieliniowości mogłyby prowadzić do błędnych wyników regulacji i uniemożliwić skuteczną kontrolę pracy pompy. Dlatego przeprowadzono szczegółowe testy obejmujące

badanie wartości dobranych dzielników napięć, wzmacniacza operacyjnego oraz testów liniowości wejść i wyjścia całego układu.

6.2.1 Badanie liniowości przetwarzanego napięcia sygnału wyjściowego ze wzmacniacza.

Proces testowania rozpoczęto od analizy funkcjonalności wzmacniacza operacyjnego LM358P, który jest kluczowym elementem odpowiedzialnym za wzmocnienie sygnału napięciowego wychodzącego z mikrokontrolera esp32. Badanie polegało na zmierzeniu napięcia wejściowego oraz wyjściowego wzmacniacza dla 10 różnych wartości w zakresie 0-10V. Wyniki pomiarów przedstawiono w tabeli 6.1. Dane prezentowane w tabeli 6.1 są zgodne z udokumentowanymi zdjęciami, które są dostępne w załączniku do pracy.

Tabela 6.1: Liniowość napięcia wyjściowego wzmacniacza operacyjnego LM358P.

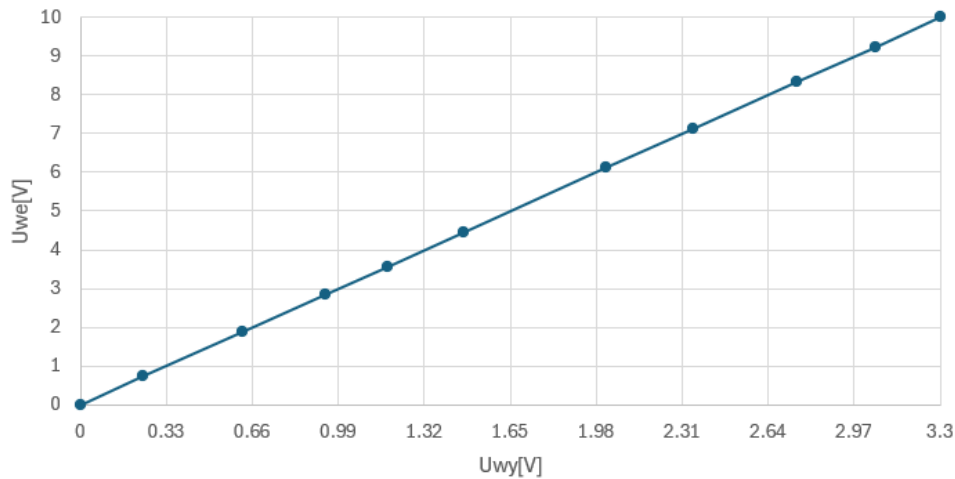
Pomiar	Napięcie wejściowe [V]	Napięcie wyjściowe [V]
0	0.00	0.00
1	0.24	0.72
2	0.62	1.87
3	0.94	2.84
4	1.18	3.57
5	1.47	4.45
6	2.02	6.12
7	2.35	7.12
8	2.75	8.33
9	3.05	9.24
10	3.30	10.00

Wykres pokazany na rysunku 6.1 świadczy o liniowości przekształcania napięcia przez wzmacniacz LM358P w zakresie 0-10V.

6.2.2 Badanie liniowości przetwarzanego napięcia sygnału z położenia kuli zaworu EV.

Kolejnym etapem testów było sprawdzenie sygnału napięciowego dla wejścia EV, aby zmniejszyć przedział wchodzącego napięcia na mikrokontroler zastosowano dzielnik napięcia. Rzeczywiste wartości rezystorów na których przeprowadzono testy przedstawiono na rysunkach 4.7a oraz 4.7b. W tym celu przeprowadzono pomiary napięcia wejściowego i odpowiadającego mu napięcia wyjściowego, które przedstawiono w tabeli 6.2.

Uzyskane dane wykazały, że sygnał napięciowy generowany przez zawór zachowuje liniowość w zakresie od 0 do 10V, co potwierdza wykres na rysunku 6.2. Liniowość ta



Rysunek 6.1: Wykres złożenia wyników pomiarów przetworzenia napięcia dla wzmacniacza.

Tabela 6.2: Pomiary napięcia wejściowego i wyjściowego dla zaworu.

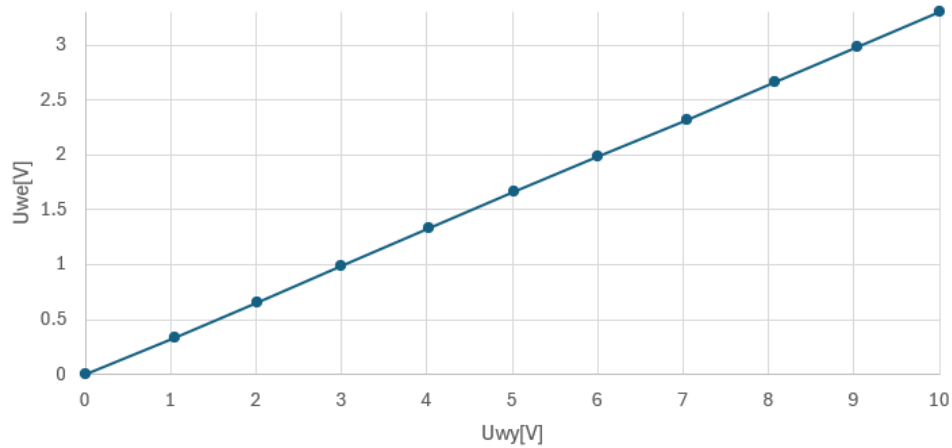
Pomiar	Napięcie wejściowe [V]	Napięcie wyjściowe [V]
0	0.00	0.00
1	1.04	0.34
2	2.01	0.66
3	3.00	0.99
4	4.03	1.33
5	5.03	1.66
6	6.00	1.98
7	7.04	2.32
8	8.08	2.66
9	9.04	2.98
10	10.00	3.30

jest kluczowa, ponieważ pozwala na dokładne odzwierciedlenie położenia kuli zaworu i precyzyjne sterowanie jego pracą przez mikrokontroler.

6.2.3 Badanie liniowości przetwarzanego napięcia sygnału BMS.

Dla wejścia z systemu zarządzania budynkiem "BMS" zastosowano identyczny dzielnik napięcia jak dla wejścia z zaworu EV. Zastosowane rezystory mają pewną tolerancję, która wynosi 5% co powoduje że, wyniki przeprowadzonych pomiarów minimalnie się różnią. Rysunki 4.7c oraz 4.7d przedstawiają wartości rezystorów używanych do przeprowadzenia badania. Wszystkie dane z przeprowadzonych pomiarów przedstawiono w tabeli 6.3.

Na podstawie danych prezentowanych w powyższej tabeli 6.3 sporządzono wykres zależności napięcia wejściowego 0-10V i napięcia wyjściowego 0-3.3V pokazanego na rysunku 6.3. Wykres świadczy o liniowości przetwarzania sygnału, na którym nam najbardziej zależy.



Rysunek 6.2: Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału z EV.

Tabela 6.3: Pomiary napięcia wejściowego i wyjściowego systemu BMS.

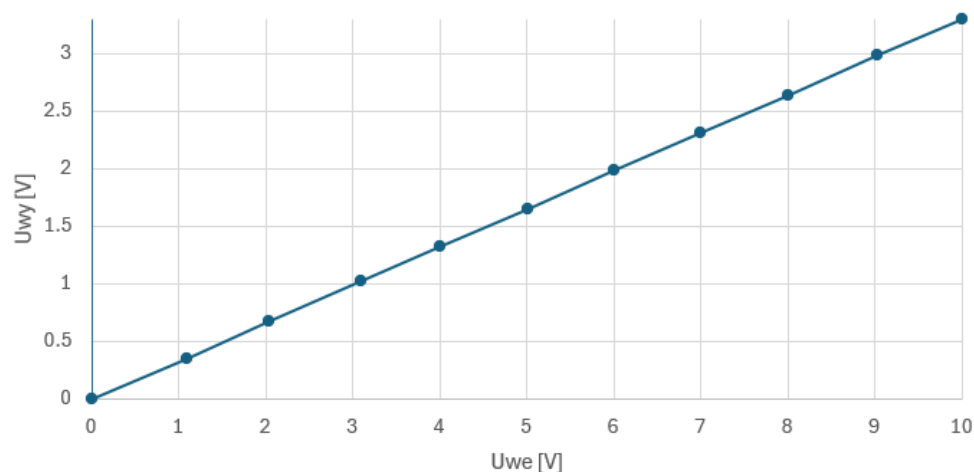
Pomiar	Napięcie wejściowe [V]	Napięcie wyjściowe [V]
0	0.00	0.00
1	1.09	0.35
2	2.04	0.67
3	3.10	1.02
4	4.01	1.32
5	5.01	1.65
6	6.00	1.98
7	7.00	2.31
8	8.01	2.64
9	9.03	2.98
10	10.00	3.30

6.3 Pomiary napięcia referencyjnego i rzeczywistego dla wejść zadajnika.

6.3.1 Pomiary napięcia referencyjnego i rzeczywistego dla EV.

Podczas testów wartości napięć odczytywanych przez mikrokontroler ESP32 za pomocą wbudowanego przetwornika analogowo-cyfrowego (ADC) zaobserwowano drobne różnice pomiędzy wartością rzeczywistą napięcia a wartością zmierzoną. Wynika to z tolerancji oraz nieliniowości charakterystycznych dla przetworników ADC w ESP32, co szczególnie widoczne jest w górnym zakresie napięcia wejściowego.

W tabeli 6.4 przedstawiono wyniki pomiarów wartości rzeczywistych napięć referencyjnych oraz wartości odczytanych przez przetwornik ADC. Dane te pokazują, że różnice między wartością referencyjną, a odczytaną zwiększają się w górnym zakresie pomiarowym, co wynika z tolerancji przetwornika.



Rysunek 6.3: Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału BMS.

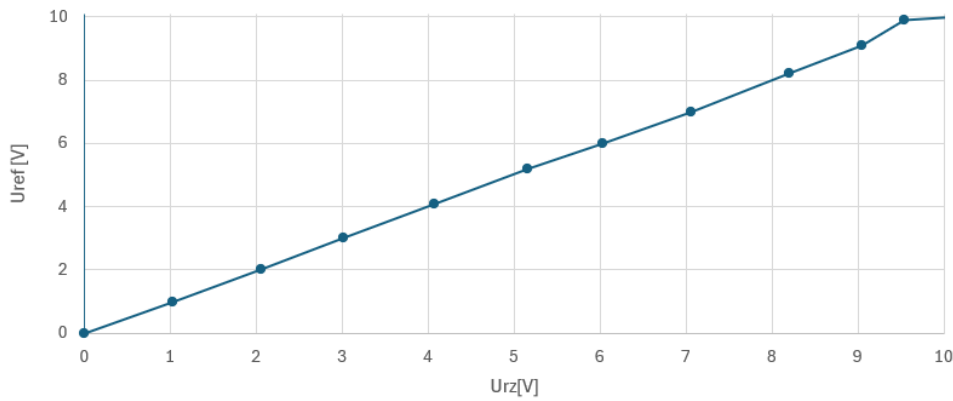
Tabela 6.4: Pomiary napięcia referencyjnego i rzeczywistego dla EV.

Pomiar	Napięcie referencyjne [V]	Napięcie rzeczywiste [V]
0	0.00	0.00
1	1.03	1.00
2	2.06	2.00
3	3.02	3.00
4	4.07	4.10
5	5.16	5.20
6	6.03	6.00
7	7.06	7.00
8	8.20	8.20
9	9.05	9.10
10	9.54	9.90
11	10.21	10.00

Na rysunku 6.4 zobrazowano zależność napięcia odczytanego przez esp32 (wartość napięcia jest przeskalowana na zakres 0-10V) od napięcia rzeczywistego. Krzywa wskazuje na poprawność działania ADC, jednak w zakresie powyżej 3.145 V można zauważyć, że przetwornik przyjmuje maksymalną wartość napięcia, dla wartość 3.145 V, ponieważ jest to najwyższy poziom przetwornika - 4096 a jak znajduje się poza tym zakresem nadpisuje wartość zmierzoną dla 3.145 czyli 10V [24]. Jest to spowodowane tolerancją zakresu przetwornika ADC.

Przetwornik ADC w ESP32 działa z rozdzielczością 12-bitową, co teoretycznie umożliwia odczyt z dokładnością 1 LSB (*Least Significant Bit*) na poziomie 3.3 V / 4096. Jednak zgodnie z dokumentacją Espressif [24], dokładność ADC może być ograniczona przez:

- **Tolerancję napięcia referencyjnego (V_{ref}):** Wartość napięcia odniesienia może się różnić między egzemplarzami mikrokontrolera.



Rysunek 6.4: Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wejściowego EV.

- **Nieliniowość charakterystyki ADC:** Widoczna szczególnie w górnym zakresie pomiarowym.
- **Zakłócenia elektromagnetyczne i szумы:** Mogą wpływać na stabilność odczytów.

Ograniczenie w odczycie powyżej 3.145 V wynika z konstrukcji przetwornika w ESP32, który posiada charakterystykę tłumienia napięcia (*attenuation*) dostosowaną do pełnego zakresu napięcia 0 – 3.3 V. W praktyce błędy w kalibracji oraz ograniczenia związane z V_{ref} prowadzą do ograniczenia zakresu odczytu [24].

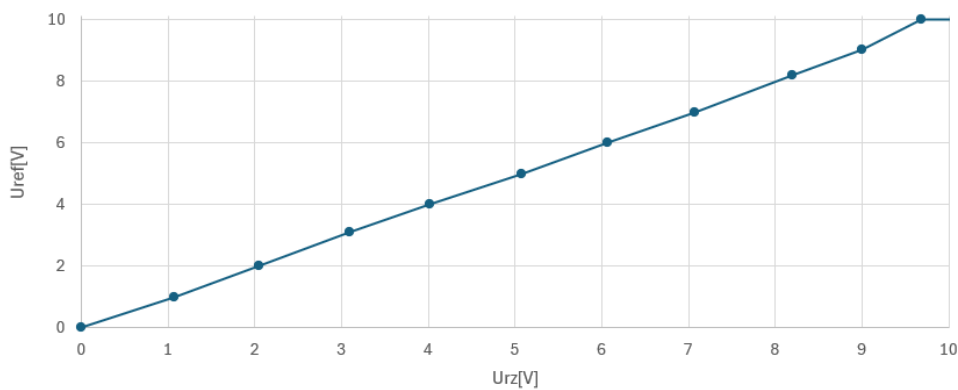
6.3.2 Pomiary napięcia referencyjnego i rzeczywistego dla wejścia z systemu zarządzania budynkiem.

Podczas testów napięć wejściowych dla wejścia z sygnału BMS zaobserwowano zjawisko analogiczne do tego opisanego w przypadku dla wejścia EV. Wartości napięć odczytywanych przez mikrokontroler ESP32 za pomocą wbudowanego przetwornika analogowo-cyfrowego (ADC) różniły się od wartości rzeczywistych w górnych zakresach pomiarowych. Przetwornik ADC poprawnie przetwarza napięcia w zakresie 0~3.145 V, ale powyżej tej wartości przypisuje maksymalną wartość, odpowiadającą limitowi przetwornika, tj. 4096 w rozdzielczości 12-bitowej [24].

W tabeli 6.5 przedstawiono wartości napięć referencyjnych i odpowiadających im napięć rzeczywistych odczytanych przez ESP32 dla wejścia BMS. Jak wynika z danych, w górnym zakresie napięcia rzeczywistego występuje różnica między napięciem wejściowym, a maksymalną wartością odczytu przetwornika. Na wykresie 6.5 zilustrowano tę zależność. Wykres potwierdza, że przetwornik ADC w ESP32 działa poprawnie w dolnym oraz środkowym zakresie pomiarowym, ale od wartości 3.145 V przypisuje maksymalną wartość odczytu równą 10V.

Tabela 6.5: Pomiary napięcia referencyjnego i rzeczywistego dla BMS.

Pomiar	Napięcie referencyjne [V]	Napięcie rzeczywiste [V]
0	0.00	0.00
1	1.07	1.00
2	2.05	2.00
3	3.09	3.10
4	4.02	4.00
5	5.07	5.00
6	6.06	6.00
7	7.07	7.00
8	8.20	8.20
9	9.00	9.00
10	9.68	10.00
11	10.03	10.00



Rysunek 6.5: Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wejściowego BMS.

6.3.3 Pomiary napięcia referencyjnego i rzeczywistego dla wyjścia.

Podczas testów przetwarzania napięcia przez wzmacniacz operacyjny zaobserwowano, że układ działa liniowo, spełniając wymagania projektowe w zakresie przetwarzania napięć od 0 V do 10 V. Liniowość potwierdzają dane przedstawione w tabeli 6.6 oraz wykres prezentowany na rysunku 6.6. Krzywa na wykresie wskazuje na proporcjonalny wzrost napięcia wyjściowego w stosunku do napięcia referencyjnego.

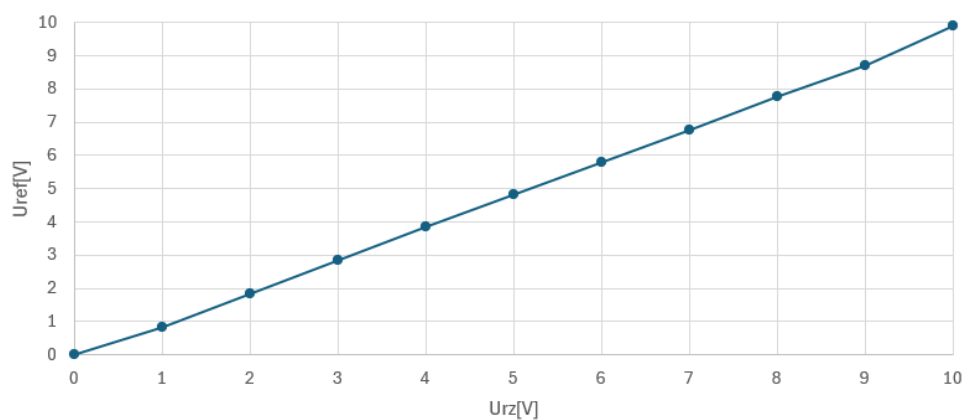
Pomimo zachowania liniowości, zauważono niewielkie różnice między napięciem referencyjnym, a rzeczywistym, wynoszące około 0.2 – 0.3 V. Odchylenia te mogą wynikać z potencjalnego przegrzania pinów i ścieżek płytki drukowanej podczas procesu lutowania, oraz tolerancji rezystorów zastosowanych w konfiguracji odpowiedniego wzmocnienia, co mogło wpłynąć na charakterystykę elektryczną połączeń. Efekt ten może powodować minimalne zmiany w wzmocnieniu sygnału przez wzmacniacz.

Pomiary wykazały, że różnice te mieszczą się w granicach akceptowalnych tolerancji

Tabela 6.6: Pomiary napięcia referencyjnego i rzeczywistego dla wzmacniacza.

Pomiar	Napięcie referencyjne [V]	Napięcie rzeczywiste [V]
0	0.00	0.00
1	1.00	0.84
2	2.00	1.84
3	3.00	2.84
4	4.00	3.86
5	5.00	4.82
6	6.00	5.81
7	7.00	6.75
8	8.00	7.76
9	9.00	8.71
10	10.00	9.90

projektowych i nie wpływają negatywnie na funkcjonalność układu. Wzmacniacz operacyjny LM358P spełnia zatem założenia projektowe.



Rysunek 6.6: Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wyjściowego.

Rozdział 7

Podsumowanie i wnioski

Podsumowując, przeprowadzone testy wykazały, że zaprojektowany układ elektornicznego optymalizatora działa zgodnie z założeniami projektowymi i spełnia wymagania dotyczące liniowości oraz precyzji przetwarzania sygnałów napięciowych. Zarówno wejścia (EV i BMS), jak i wyjście wzmacniacza operacyjnego LM358P, charakteryzują się wysoką liniowością, co potwierdzają uzyskane wyniki pomiarów. Przetwornik ADC w ESP32 jest ograniczony do zakresu $0 - 3.145\text{ V}$, został przeskalowany do zakresu $0 - 10\text{ V}$, co w rzeczywistości pozwoliło na odczytanie wartości w zakresie od 0 do 9.53 V . W przyszłości ten problem można rozwiązać stosując inny przetwornik na peryferiach płytki drukowanej.

Na wyjściu wzmacniacza zaobserwowano niewielkie różnice pomiędzy napięciami rzeczywistymi a referencyjnymi, wynoszące około $0.2 - 0.3\text{ V}$, co może wynikać z przegrzania pinów i ścieżek podczas lutowania oraz tolerancji rezystorów. Różnice te jednak mieszczą się w granicach akceptowalnych błędów, ponieważ wzrost napięcia sterującego pompą zachowuje liniowość, która była pożądana. Projekt spełnia założenia dotyczące precyzji oraz stabilności działania i może być wdrożony w docelowym środowisku. W przyszłości można go rozwinąć, optymalizując proces montażu i lutowania w celu zminimalizowania potencjalnych błędów, a także zastosować zewnętrzny przetwornik ADC o szerszym zakresie pracy, co pozwoli na bardziej precyzyjne odczyty w pełnym zakresie napięć $0 - 10\text{ V}$. Projekt ma duży potencjał rozwojowy i może być dostosowywany do bardziej wymagających zastosowań w systemach sterowania. Projekt można rozbudować, wykorzystując moduł Wi-Fi zastosowanego w platformie ESP32 do zapisu danych w przemysłowej bazie danych - influxDB. Integracja z InfluxDB mogłaby obejmować zapisywanie parametrów pracy pompy w czasie rzeczywistym, co pozwoliłoby na dokładniejszą diagnostykę systemu. A następnie można utworzyć zdalny system monitorowania i sterowania pompą poprzez interfejs webowy lub aplikację mobilną.

Bibliografia

- [1] Adafruit. *RTCLib: Real-Time Clock Library*. 2024. URL: <https://github.com/adafruit/RTCLib> (term. wiz. 22.12.2024).
- [2] All3DP. *PETG Filament: All You Need to Know*. 2024. URL: <https://all3dp.com/2/petg-filament-all-you-need-to-know/> (term. wiz. 22.12.2024).
- [3] Autodesk. *Autodesk Inventor Documentation*. Autodesk Press, 2024.
- [4] Autodesk. *EAGLE PCB Design Software Documentation*. 2024. URL: <https://www.autodesk.com/products/eagle/overview> (term. wiz. 22.12.2024).
- [5] Belimo. *Energy Valve with TEM - Dokumentacja Produktu*. n.d. URL: https://www.belimo.com/pl/pl_PL/products/sensors/product-documentation/energy-valve-with-tem (term. wiz. 27.12.2024).
- [6] W. Bolton. *Podstawy Elektrotechniki. Tom 1*. Warszawa: Wydawnictwo Naukowe PWN, 2017.
- [7] Botland. *Czujnik obrotu - enkoder 20 impulsów Okystar KY-040*. 2025. URL: <https://botland.com.pl/enkodery/16558-czujnik-obrotu-enkoder-20-impulsow-okystar-ky-040-5904422366278.html> (term. wiz. 02.01.2025).
- [8] Botland. *Dioda LED 5mm zielona (10 szt.)* 2025. URL: <https://botland.com.pl/diody-led/19994-dioda-led-5mm-zielona-10szt-justpi-5904422328825.html> (term. wiz. 02.01.2025).
- [9] Botland. *ESP32 WiFi + BT 4.2 Platforma z modulem ESP-WROOM-32 zgodny z ESP32-DEVKIT*. 2025. URL: <https://botland.com.pl/moduly-wifi-i-bt-esp32/8893-esp32-wifi-bt-42-platforma-z-modulem-esp-wroom-32-zgodny-z-esp32-devkit-5904422337438.html> (term. wiz. 02.01.2025).
- [10] Botland. *Wzmacniacz operacyjny LM358PN (5 szt.)* 2025. URL: <https://botland.com.pl/wzmacniacze-operacyjne/9880-wzmacniacz-operacyjny-lm358pn-5-szt-5904422309374.html> (term. wiz. 02.01.2025).
- [11] Botland. *Zestaw przewodów połączeniowych JustPi 20cm 3x40 szt. M-M, Ż-Ż, M-Ż (120 szt.)* 2025. URL: <https://botland.com.pl/przewody-polaczeniowe/19946-zestaw-przewodow-polaczeniowych-justpi-20cm-3x40szt-m-m-z-z-m-z-120szt-5904422328702.html> (term. wiz. 02.01.2025).

-
- [12] Brak informacji o autorze. *Multimetr cyfrowy Sonel CMM-10*. Dostęp online. 2024. URL: <https://sonel.pl/pl/produkt/multimetr-cyfrowy-sonel-cmm-10/> (term. wiz. 23.12.2024).
- [13] Brak informacji o autorze. *Zasilacz laboratoryjny DF173005C*. Dostęp online. 2024. URL: <https://ndn.com.pl/pl/zasilacze-laboratoryjne/3982-df173005c-ndn.html> (term. wiz. 23.12.2024).
- [14] Last Minute Engineers. *Rotary Encoder KY-040 with Arduino*. 2024. URL: <https://lastminuteengineers.com/rotary-encoder-arduino-tutorial/> (term. wiz. 22.12.2024).
- [15] Texas Instruments. *LM358P Dual Operational Amplifier Datasheet*. 2024. URL: <https://www.ti.com/lit/ds/symlink/lm358.pdf> (term. wiz. 22.12.2024).
- [16] Oliver Kraus. *U8g2: Library for Monochrome Displays*. 2024. URL: <https://github.com/olikraus/u8g2> (term. wiz. 22.12.2024).
- [17] Bambu Lab. *Bambu Lab A1 User Manual*. 2024. URL: <https://www.bambulab.com/> (term. wiz. 22.12.2024).
- [18] NEO-TOOLS. *Instrukcja obsługi miernika cyfrowego NEO-TOOLS 94-001*. Dostęp online: instrukcja PDF. 2024. URL: <https://www.manualslib.com/manual/2086582/Neo-94-001.html> (term. wiz. 23.12.2024).
- [19] Nieznany. *Wzmacniacz operacyjny — wstęp*. 2024. URL: https://home.agh.edu.pl/~ggora/PE/lab11_12/WzmacniaczOperacyjny_Wstep.pdf (term. wiz. 23.12.2024).
- [20] Botland Sp. z o.o. *RTC DS1307 + 32kb EEPROM 24C32 I2C - zegar czasu rzeczywistego z pamięcią*. 2024. URL: <https://botland.com.pl/moduly-rtc/12507-rtc-ds1307-32kb-eeeprom-24c32-i2c-zegar-czasu-rzeczywistego-z-pamiecia-5904422341121.html> (term. wiz. 22.12.2024).
- [21] Botland Sp. z o.o. *Wyświetlacz OLED niebieski graficzny 1.3"128x64px I2C V2 - niebieskie znaki*. 2024. URL: <https://botland.com.pl/wyswietlacze-oled/8246-wyswietlacz-oled-niebieski-graficzny-13-128x64px-i2c-v2-niebieskie-znaki-5904422311339.html> (term. wiz. 22.12.2024).
- [22] Sankom. *Wilo-Yonos ECO-BMS Katalog*. n.d. URL: http://www.sankom.pl/download/dk/documentation/wilo/wilo-yonos%20eco-bms_katalog.pdf (term. wiz. 27.12.2024).

- [23] Espressif Systems. *Analog to Digital Converter (ADC) Calibration Driver - ESP32*. 2024. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc_calibration.html (term. wiz. 22.12.2024).
- [24] Espressif Systems. *ESP-IDF ADC Characteristics and Calibration*. 2024. URL: <https://docs.espressif.com/projects/esp-idf/en/release-v4.4/esp32/api-reference/peripherals/adc.html> (term. wiz. 27.12.2024).
- [25] Espressif Systems. *ESP32-WROOM-32 Data Sheet*. 2024. URL: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf (term. wiz. 22.12.2024).
- [26] Monolithic Power Systems. *MP2315 3.5A, 24V, 1.4MHz Step-Down Converter*. 2024. URL: https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP2315/document_id/1285/ (term. wiz. 22.12.2024).
- [27] Arduino Team. *Arduino Language Reference*. 2024. URL: <https://www.arduino.cc/reference/en/> (term. wiz. 22.12.2024).
- [28] Arduino Team. *Wire - Arduino Reference*. 2024. URL: <https://reference.arduino.cc/reference/en/language/functions/communication/wire/> (term. wiz. 22.12.2024).

Dodatki

Spis skrótów i symboli

BMS Systemu zarządzania budynkiem (ang. *Building Management System*)

EV Zawór Energetyczny (ang. *Energy Valve*)

ADC przetwornik analogowo-cyfrowy (ang. *Analog to Digital Converter*)

OLED organiczna dioda elektroluminescencyjna (ang. *Organic Light Emitting Diode*)

RTC Zegar czasu rzeczywistego (ang. *real-time clock*)

LED dioda elektroluminescencyjna (ang. *light-emitting diode*)

IoT Internet rzeczy (ang. *internet of things*)

PCB Płytką drukowana (ang. *Printed Circuit Board*)

HVAC ogrzewanie, wentylacja i klimatyzacja (ang. *Heating, Ventilation i Air Conditioning*)

V volt

μ F mikro Fahrad

I2C szeregową, dwukierunkową magistralą (ang. *Inter-Integrated Circuit*))

int typ całkowity (ang. *integer*))

bool typ logiczny (ang. *boolean*))

float typ zmiennie przecinkowy (ang. *floating point*))

GPIO Wejście-wyjście ogólnego przeznaczenia (ang. *General Purpose Input Output*))

MHz milion herców (ang. *megaherc*)

PLC Programowalny Sterownik Logiczny (ang. *Programmable Logic Controller*)

IDE zintegrowane środowisko programistyczne (ang. *Integrated Development Environment*)

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- schematic.sch
- schematic.brd
- main.ino
- Top.stl
- Bottom.stl
- udokumentowane_zdjęcia_pomiarów.zip

Spis rysunków

2.1	Ideowy schemat działania zadajnika (rysunek od "zleceniodawcy").	6
2.2	Opis sygnałów i wyprowadzeń pompy obiegowej Wilo-Yonos ECO 25 [22]. .	7
2.3	Wyprowadzenia sygnałów z zaworu Belimo Energy Valve™ [5].	8
3.1	Zasilacz laboratoryjny DF173005C NDN – 30V, 5A. Zdjęcie ze strony sklepu ndn.com.pl [13].	12
3.2	Multimetr Sonel CMM-10.	13
3.3	Multimetr NEO 94-001.	14
4.1	Schemat połączeń zadajnika zaprojektowany w programie Autodesk Eagle.	17
4.2	ESP32 DevKit v1 - zdjęcie ze strony sklepu Botland [9].	17
4.3	Moduł RTC DS1307 z pamięcią EEPROM 24C32 - zdjęcie ze strony sklepu Botland [20].	18
4.4	Wyświetlacz OLED - zdjęcie ze strony sklepu Botland [21].	18
4.5	Encoder obrotowy KY-040 - zdjęcie ze strony sklepu Botland [7].	19
4.6	Dzielnik napięcia.	21
4.7	Zmierzone wartości rezystancji rezystorów omomierzem.	22
4.8	Wzmacniacz operacyjny w konfiguracji wzmacniacza nieodwracającego. . .	23
4.9	Zmierzone wartości rezystancji rezystorów omomierzem.	24
4.10	Model zaprojektowanej płytki PCB.	28
4.11	Płytką drukowaną z przylutowanymi komponentami.	29
4.12	Model zaprojektowanej obudowy w programie Inventor.	30
4.13	Końcowy wygląd zaprojektowanego zadajnika.	32
5.1	Wykorzystane biblioteki.	34
5.2	Definicja pinów mikrokontrolera.	36
5.3	Definicja wyświetlacza OLED.	37
5.4	Definicja RTC.	38
5.5	Typ wyliczeniowy ENUM.	38
5.6	Korekta offsetu napięcia dla wejść zadajnika.	39
5.7	Funkcja interpolacji kawałkowej.	41
5.8	Funkcja wygładzania eksponencjalnego.	41

5.9	Funkcja zmiany trybu pracy systemu.	43
5.10	Funkcja obsługi encodera BMS.	44
5.11	Funkcja obsługi encodera VOLT.	44
5.12	Funkcja obsługi encodera TIME.	45
5.13	Inicjalizacja komunikacji.	46
5.14	Sprawdzenie poprawności działania modułu RTC.	46
5.15	Synchronizacja czasu modułu RTC.	47
5.16	Inicjalizacja wyświetlacza OLED	47
5.17	Konfiguracja trybów pracy pinów mikrokontrolera.	48
5.18	Inicjalizacja stanów początkowych encoderów.	48
5.19	Konfiguracja przerw dla encoderów.	49
5.20	Inicjalizacja początkowego stanu wyjściowego pompy	49
5.21	Obsługa przycisku z eliminacją drgań styków (<i>debouncing</i>).	50
5.22	Odczyt, przetwarzanie i skalowanie napięć odczytanych z pinów: VP i VN.	51
5.23	Logika regulacji w trybie automatycznym.	52
5.24	Logika regulacji w trybie ręcznym.	53
5.25	Obsługa diod sygnalizacyjnych.	53
5.26	Wyświetlanie informacji na wyświetlaczu OLED.	54
5.27	Wyświetlanie informacji na wyświetlaczu OLED.	55
5.28	Realizacja operacji w regularnych odstępach czasu.	55
6.1	Wykres złożenia wyników pomiarów przetworzenia napięcia dla wzmacniacza.	59
6.2	Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału z EV.	60
6.3	Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału BMS.	61
6.4	Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wejściowego EV.	62
6.5	Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wejściowego BMS.	63
6.6	Wykres złożenia wyników pomiarów przetworzenia napięcia dla sygnału wyjściowego.	64

Spis tabel

4.1	Kosztorys komponentów	20
6.1	Liniowość napięcia wyjściowego wzmacniacza operacyjnego LM358P.	58
6.2	Pomiary napięcia wejściowego i wyjściowego dla zaworu.	59
6.3	Pomiary napięcia wejściowego i wyjściowego systemu BMS.	60
6.4	Pomiary napięcia referencyjnego i rzeczywistego dla EV.	61
6.5	Pomiary napięcia referencyjnego i rzeczywistego dla BMS.	63
6.6	Pomiary napięcia referencyjnego i rzeczywistego dla wzmacniacza.	64